

Project Introduction: Security Analysis & Vulnerability Auditing of DVWA

1. What is DVWA (Damn Vulnerable Web Application)?

Damn Vulnerable Web Application (DVWA) is a PHP/MySQL-based web application designed intentionally with security flaws. It serves as a legal, highly interactive training ground for security professionals, ethical hackers, and developers.

Unlike real-world applications that strive to mask their flaws, DVWA categorizes vulnerabilities into distinct, toggleable security levels:

- **Low:** No sanitization or defensive coding; raw parameters are passed directly to database or system shells.
- **Medium:** Basic, flawed validation rules are implemented (e.g., blacklisting or simple character escaping), simulating realistic, poorly-configured production code.
- **High:** Advanced mitigation techniques are implemented, often requiring complex payload formatting or secondary vulnerability chaining to bypass.
- **Impossible:** Fully secure, secure-coded templates demonstrating how the backend should ideally be structured to prevent exploitation altogether.

2. Methodology: Reviewing Code Design and Architecture

In this security audit, we bypassed automated scanner tools to manually analyze the underlying codebase and design patterns of the application. By reviewing the raw PHP source files alongside database schemas, we mapped how poor system design directly leads to active exploit paths.

Our review of the design architecture yielded two primary critical insights:

A. The "Input-to-Sink" Design Flaw

We discovered that the core architectural vulnerability across several modules in DVWA is the lack of isolation between user inputs and execution sinks. User input is accepted directly from HTTP GET or POST parameters and immediately processed by critical system sinks (such as `eval()`, database query engines `mysqli_query()`, or system shell executors like `shell_exec()`) without strict validation boundaries.

B. Defensive Logic Placed in the Client-Side (UI)

In multiple instances (specifically within the JavaScript validation modules), we identified a critical architectural design flaw: security policies and input validations were implemented on the client-side rather than on the server. Because an attacker can easily bypass client-side restrictions by intercepting and modifying the raw HTTP requests via proxies, this design flaw effectively renders client-side sanitization useless.

3. Vulnerability Code Deep-Dive & Target Modules

Code Vulnerability :

1. Vulnerability Report: SQL Injection Vulnerability :

Phase: Discovery & Identification :

Action: We navigated to the "SQL Injection" page in the DVWA application. We performed an initial test by entering 1 into the input field.

Observation: The application successfully retrieved the user's name and surname, confirming the page was interacting with the database

Phase: Vulnerability Exploitation (Before Fix) :

Action: To verify if the field was vulnerable, we entered the payload ' OR '1'='1 into the User ID field.

Result: The application displayed all records in the database, proving that the input was not being validated and that SQL injection was possible.

Vulnerable Code Analysis: The original code was insecure because it directly combined user input with the SQL query.

Vulnerable Code Snippet:

PHP

```
$query = "SELECT first_name, last_name FROM users WHERE user_id =  
'$id'"; $result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

Why it was Vulnerable: The code treated the user's input as part of the command itself. Because '1'='1' is always true, the database ignored the ID requirement and dumped the entire user table.

Phase: Remediation (The Fix)

Action: We updated the source code to implement "Prepared Statements." This approach separates the SQL command from the user-provided data.

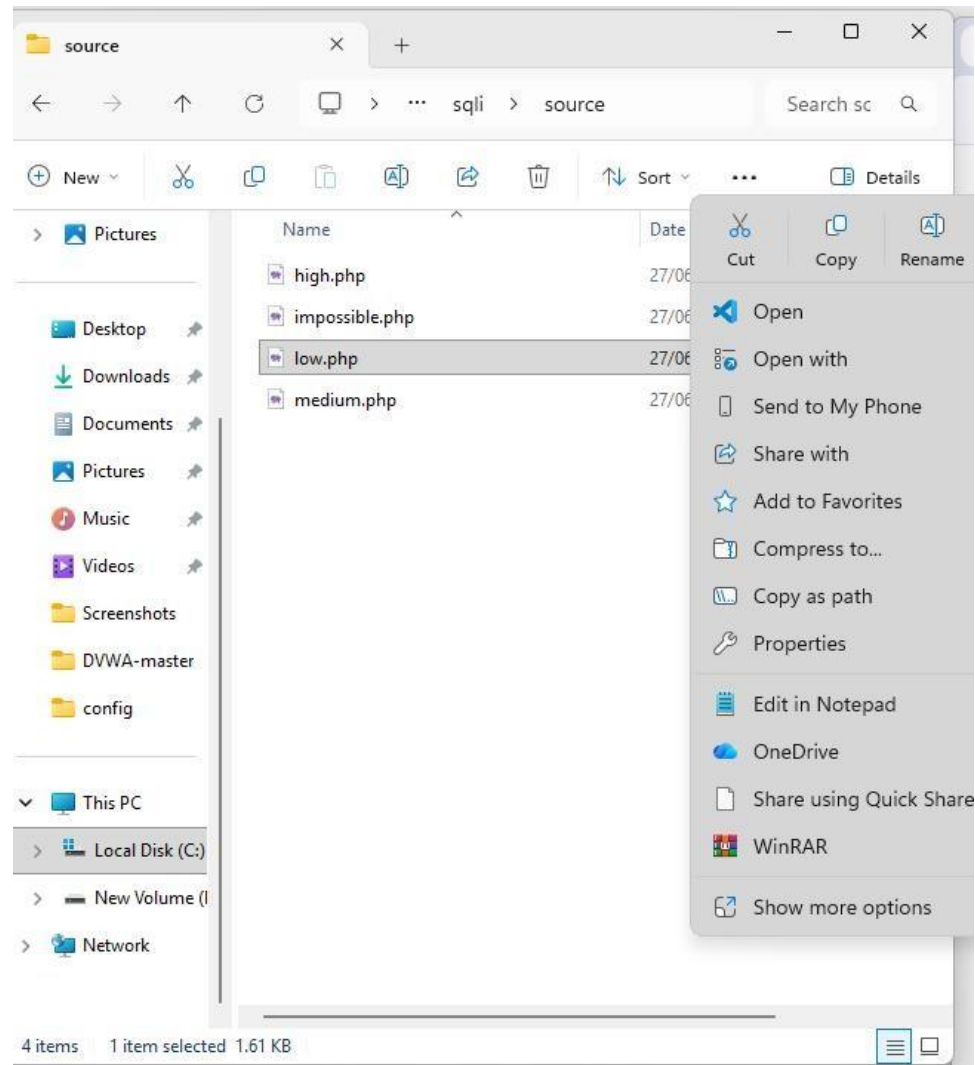
Fixed Code Snippet:

PHP

```
$stmt = mysqli_prepare($GLOBALS["__mysqli_ston"], $query);  
mysqli_stmt_bind_param($stmt, "s", $id);  
mysqli_stmt_execute($stmt); $result = mysqli_stmt_get_result($stmt);
```

Why it is Secure: By using `mysqli_prepare()`, we defined the query structure first. The `mysqli_stmt_bind_param()` function then tells the database to treat the input (`$id`) only as a literal string (data), never as an executable command

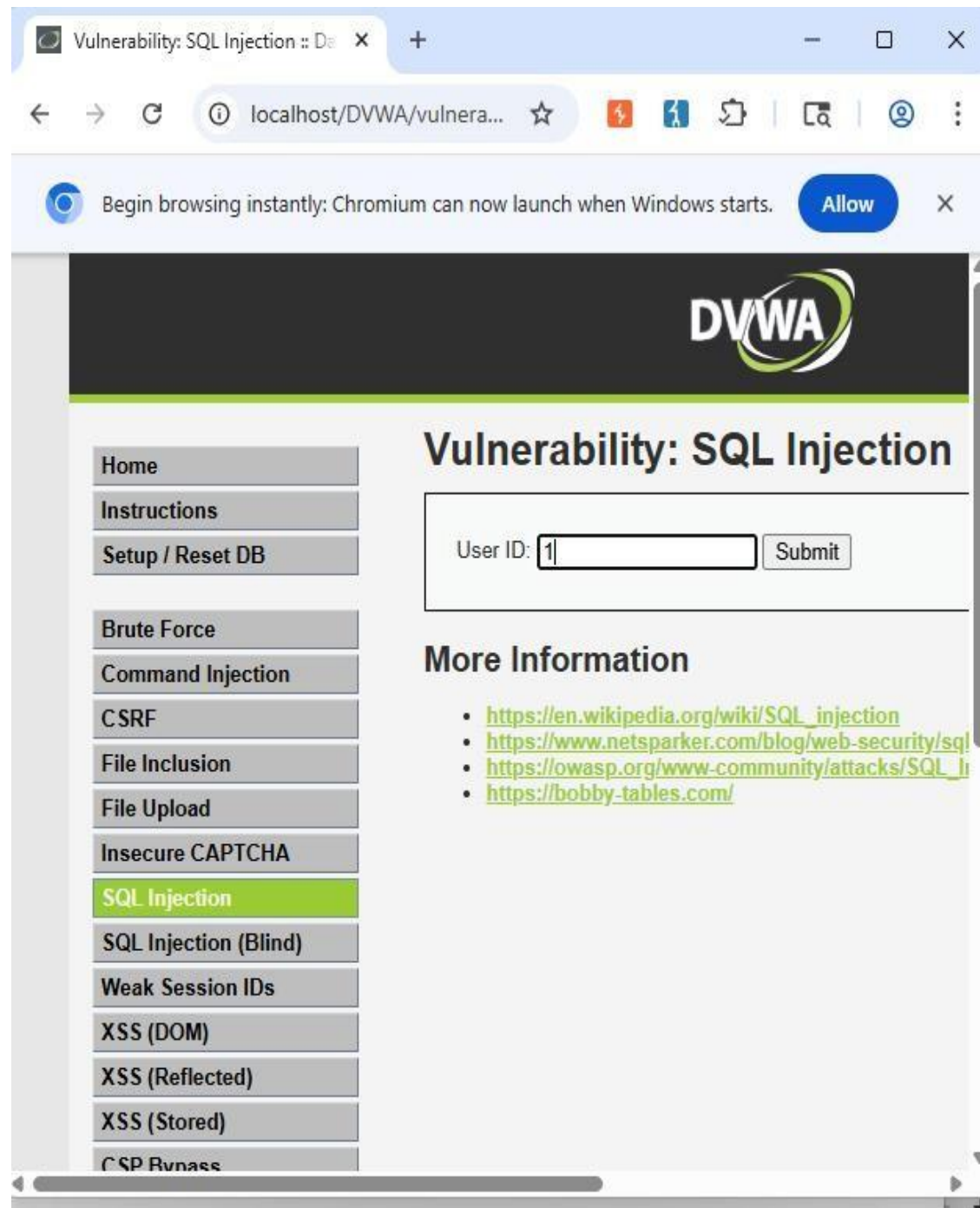
STEP NO 1 : FIND WEAK POINT



STEP 2 : BEFORE FIX CODE :

```
// VULNERABLE CODE - Yahan koi protection nahi hai
$query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
$result = mysqli_query($GLOBALS["__mysqli_ston"], $query);
```

STEP 3 : BEFORE FIX :



XAMPP Control Panel v3.3.0 [Compiled: Apr 6th 2021]

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Request to http://localhost:8080

Time	Type	Direction	Method	URL
16:49:46...	HTTP	→ Request	GET	http://localhost/DVWA/vulnerabilities/sql/?id=1&Submit=Submit&user_token=b021e13de34f4fc10850e2ad8208606

Request

Pretty Raw Hex

```
1 GET /DVWA/vulnerabilities/sql/?id=1&Submit=Submit&user_token=b021e13de34f4fc10850e2ad8208606 HTTP/1.1
2 Host: localhost
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: ?0
5 sec-ch-ua-platform: "Windows"
6 Accept-Language: en-US,en;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.0
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
```

Vulnerability: SQL Injection = 0

localhost/DVWA/vulnera...

Begin browsing instantly; Chromium can now launch when Windows starts. Allow

Vulnerability: SQL Injection

Reset DB

User ID:

ID: 1
First name: admin
Surname: admin

More Information

- https://en.wikipedia.org/wiki/SQL_injection
- <https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/>
- https://owasp.org/www-community/attacks/SQL_injection
- <https://bobby-tables.com/>

Activate Windows
Go to Settings to activate Windows.

4:49 pm
27/06/2026

← → ↻ ⓘ localhost/DVWA/vulnera... ☆ ⚡ 📄 🗑️ 🔍 👤

Begin browsing instantly: Chromium can now launch when Windows starts. **Allow**



Vulnerability: SQL Injection

- Home
- Instructions
- Setup / Reset DB
- Brute Force
- Command Injection
- CSRF
- File Inclusion
- File Upload
- Insecure CAPTCHA
- SQL Injection**
- SQL Injection (Blind)
- Weak Session IDs
- XSS (DOM)
- XSS (Reflected)
- XSS (Stored)

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

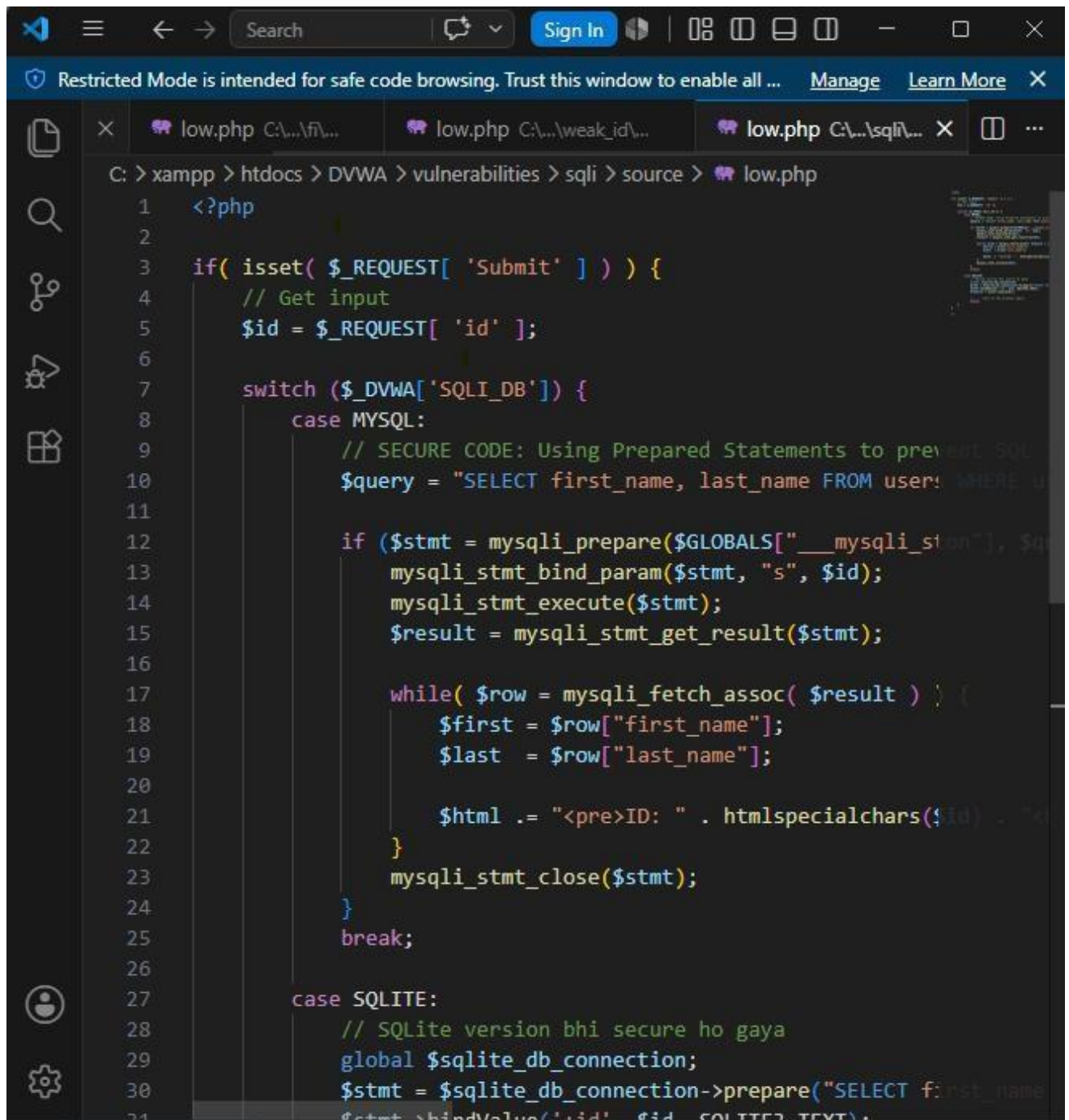
ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

Activate Windows

STEP 4 : FIX CODE :



Burp Suite Community Edition										
Burp Project Intruder Repeater View Help										
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comp										
Intercept HTTP history WebSockets history Match and replace Proxy settings										
Filter settings: Hiding CSS and image content; hiding specific extensions										
#	...	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Titl
38	...	GET	/DVWA/vulnerabilities/sqli/			200	5243	HTML		Vu
39	...	GET	/DVWA/vulnerabilities/sqli/?id=1&...	✓	✓	200	5053	HTML		Vu
18	...	GET	/DVWA/vulnerabilities/sqli/?id=1&...	✓		200	5112	HTML		Vu
28	...	GET	/DVWA/vulnerabilities/sqli/?id=1&...	✓		200	5112	HTML		Vu
29	...	GET	/DVWA/vulnerabilities/sqli/?id=1&...	✓		302	387	HTML		Vu
15	...	GET	/DVWA/vulnerabilities/sqli/			200	5053	HTML		Vu
7	...	POST	/DVWA/login.php	✓		302	526	HTML	php	Vu
21	...	GET	/DVWA/vulnerabilities/sqli/			200	5053	HTML		Vu

STEP NO 5 : AFTER FIX CODE :

The screenshot shows two windows. On the left is the Burp Suite interface, and on the right is a web browser displaying the DVWA (Damn Vulnerable Web Application) page.

Burp Suite Interface:

- Proxy Tab:** Shows the 'Intercept on' button and a list of intercepted requests. The selected request is a GET request to `/DVWA/vulnerabilities/sqli/?id=1&OR=1&Submit=Submit`.
- Request Tab:** Displays the raw HTTP request details:


```
GET /DVWA/vulnerabilities/sqli/?id=1&OR=1&Submit=Submit HTTP/1.1
Host: localhost
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
```

Web Browser (localhost/DVWA...):

- The page title is "Vulnerability: SQL Injection".
- The "User ID" input field contains the payload `"OR"1=1`.
- The "Submit" button is visible.
- Under "More Information", there are links to resources about SQL injection:
 - https://en.wikipedia.org/wiki/SQL_injection
 - <https://www.netsparker.com/blog/web-security/sql-injection/>
 - https://owasp.org/www-community/attacks/SQL_injection/
 - <https://bobby-tables.com/>

2 : Vulnerability Report: Command Injection :

Phase: Discovery & Identification

Action: We navigated to the "Command Injection" page in DVWA. We entered a valid IP address (127.0.0.1) in the input field to test functionality.

Observation: The application successfully executed the ping command and displayed the network statistics, confirming that user input was being passed to the system shell.

Phase: Vulnerability Exploitation (Before Fix)

Action: We tested for command injection by appending an additional OS command using the && operator: 127.0.0.1 && dir.

Result: The application executed both the ping command and the dir command, listing the directory contents. This confirmed that the application was vulnerable to OS Command Injection.

Vulnerable Code Analysis: The original code was insecure because it directly passed the raw user input to the shell_exec() function

Vulnerable Code Snippet:

```
PHP
$target = $_REQUEST['ip'];$cmd = shell_exec('ping ' . $target);
```

Why it was Vulnerable: The application did not sanitize or validate the \$target input. By using shell operators (like &&, ;, or |), an attacker can terminate the intended command and inject their own malicious system commands to run with the application's privileges.

Phase: Remediation (The Fix)

Action: We implemented Input Validation to ensure only properly formatted IP addresses are accepted by the application.

Fixed Code Snippet:

```
PHP
$target = $_REQUEST['ip'];if (filter_var($target, FILTER_VALIDATE_IP)) {
    // Safe to execute ping
    $cmd = shell_exec('ping ' . $target);
} else {
    $html .= "<pre>Invalid IP address!</pre>";
}
```

Why it is Secure: By using the filter_var(\$target, FILTER_VALIDATE_IP) function, the application now checks if the input is a valid IP address before executing

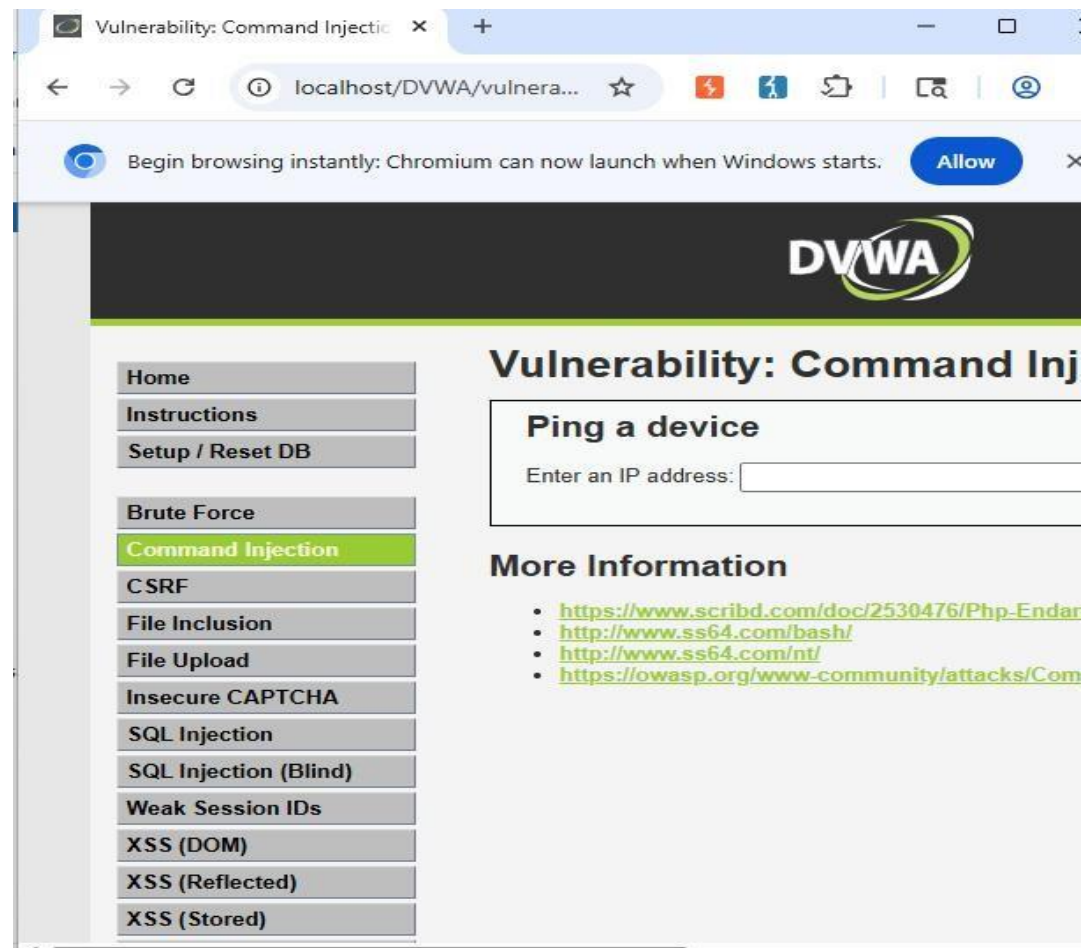
the command. If a user tries to inject malicious characters (like &&), the filter fails, and the command is never executed.

Phase: Verification (Re-check)

Action: After applying the fix, we performed a re-check by inputting the same malicious payload (127.0.0.1 && dir).

Final Result: The application rejected the input, displaying an "Invalid IP address!" error instead of executing the dir command. This confirmed the vulnerability is successfully patched.

STEP NO 1 : FIND WEAK POINT



STEP 2 : BEFORE FIX :

Intercept on Forward Drop

Request POST http://localhost/DVWA/vulnerabilities/exec/

Request

Pretty Raw Hex

```
1 POST /DVWA/vulnerabilities/exec/ HTTP/1.1
2 Host: localhost
3 Content-Length: 37
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
```

Event log [39] All issues Memory: 149.6MB of 3.93GB

Vulnerability: Command Injection

Ping a device

Enter an IP address: 127.0.0.1 && dir Submit

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Exec>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Intercept on Forward Drop

Intercept is on

Messages between Burp's browser and your target servers are held here. This enables you to analyze and modify these messages, before you forward them.

Learn more Open browser

Vulnerability: Command Injection

Ping a device

Enter an IP address: Submit

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms
Volume in drive C has no label.
Volume Serial Number is 4C70-31D7

Directory of C:\xampp\htdocs\DVWA\vulnerabilities\exec

27/06/2026 03:16 am

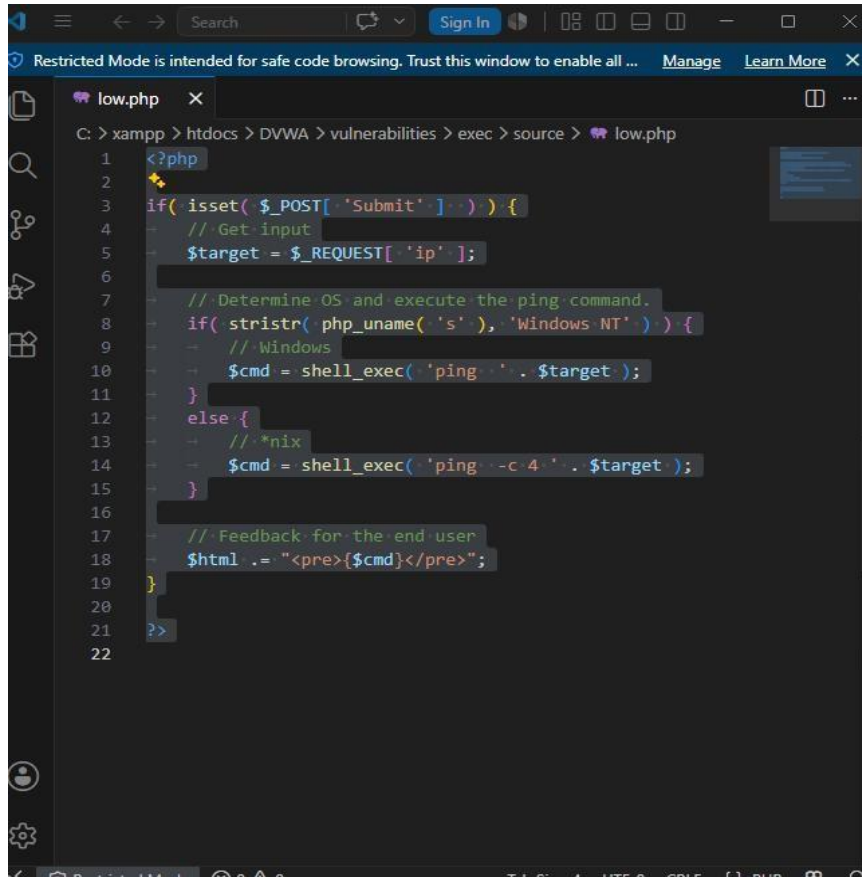
27/06/2026 03:16 am help

27/06/2026 03:07 am 1,829 index.php

27/06/2026 03:16 am source

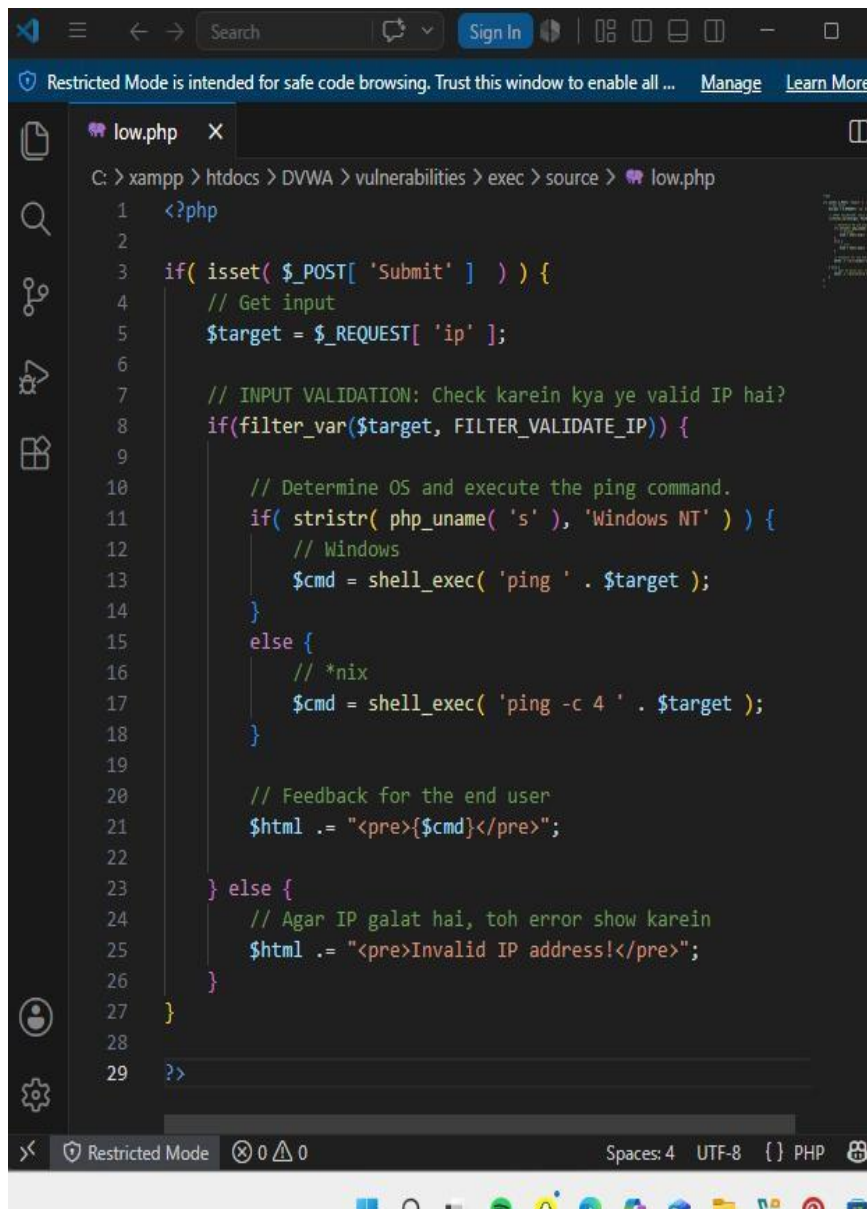
1 File(s) 1,829 bytes
4 Dir(s) 75,725,279,232 bytes free

STEP 3 : BEFORE FIX CODE :



```
1 <?php
2
3 if( isset($_POST['Submit']) ) {
4     //Get input
5     $target = $_REQUEST['ip'];
6
7     // Determine OS and execute the ping command.
8     if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
9         // Windows
10        $cmd = shell_exec( 'ping -n 1 ' . $target );
11    }
12    else {
13        // *nix
14        $cmd = shell_exec( 'ping -c 4 ' . $target );
15    }
16
17    // Feedback for the end user
18    $html .= "<pre>{$cmd}</pre>";
19 }
20
21 ?>
22
```

STEP 4 : FIX CODE :



```
1 <?php
2
3 if( isset( $_POST[ 'Submit' ] ) ) {
4     // Get input
5     $target = $_REQUEST[ 'ip' ];
6
7     // INPUT VALIDATION: Check karein kya ye valid IP hai?
8     if(filter_var($target, FILTER_VALIDATE_IP)) {
9
10        // Determine OS and execute the ping command.
11        if( striistr( php_uname( 's' ), 'Windows NT' ) ) {
12            // Windows
13            $cmd = shell_exec( 'ping ' . $target );
14        }
15        else {
16            // *nix
17            $cmd = shell_exec( 'ping -c 4 ' . $target );
18        }
19
20        // Feedback for the end user
21        $html .= "<pre>{$cmd}</pre>";
22
23    } else {
24        // Agar IP galat hai, toh error show karein
25        $html .= "<pre>Invalid IP address!</pre>";
26    }
27 }
28
29 ?>
```

STEP NO.5 : AFTER FIX CODE :

Image showing Burp Suite and a web browser displaying the DVWA 'Vulnerability: Command Injection' page. The browser shows a successful ping to 127.0.0.1.

Burp Suite Request:

```
1 POST /DVWA/vulnerabilities/exec/ HTTP/1.1
2 Host: localhost
3 Content-Length: 26
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
```

Web Browser Output:

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Pinging 127.0.0.1 with 32 bytes of data:
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128
Reply from 127.0.0.1: bytes=32 time<1ms TTL=128

Ping statistics for 127.0.0.1:
Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
Minimum = 0ms, Maximum = 0ms, Average = 0ms

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

Image showing Burp Suite and a web browser displaying the DVWA 'Vulnerability: Command Injection' page. The browser shows an error message: 'Invalid IP address!'.

Burp Suite Request:

```
1 POST /DVWA/vulnerabilities/exec/ HTTP/1.1
2 Host: localhost
3 Content-Length: 37
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
```

Web Browser Output:

Vulnerability: Command Injection

Ping a device

Enter an IP address:

Invalid IP address!

More Information

- <https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution>
- <http://www.ss64.com/bash/>
- <http://www.ss64.com/nt/>
- https://owasp.org/www-community/attacks/Command_Injection

.....

3: Vulnerability Report: Reflected Cross-Site Scripting (XSS-R)

1. Phase: Discovery & Identification

Action: We navigated to the "XSS (Reflected)" module. We entered a test string in the input field, such as Laiba, and observed that the input was reflected directly in the page response.

Observation: We then injected a JavaScript payload: `<script>alert('XSS-R')</script>`. The browser executed the script immediately upon the page reloading, confirming a Reflected XSS vulnerability.

2. Phase: Vulnerability Analysis (Before Fix)

Result: The application takes the input from the URL parameters and embeds it directly into the HTML response without any filtering.

Vulnerable Code Snippet:

```
PHP
// The application takes input directly from the URL
echo '<pre>Hello '.
$_GET['name'] . '</pre>';
```

Why it was Vulnerable: Because the application "reflects" the user input back into the page code without sanitization, an attacker can craft a malicious link. When an unsuspecting user clicks that link, the script runs in their browser.

3. Phase: Remediation (The Fix)

Action: We applied input sanitization to ensure the browser treats the input as text, not as HTML/code.

Fixed Code Snippet:

```
PHP
// Use htmlspecialchars to neutralize the input
echo '<pre>Hello '.
htmlspecialchars($_GET['name'], ENT_QUOTES) . '</pre>';
```

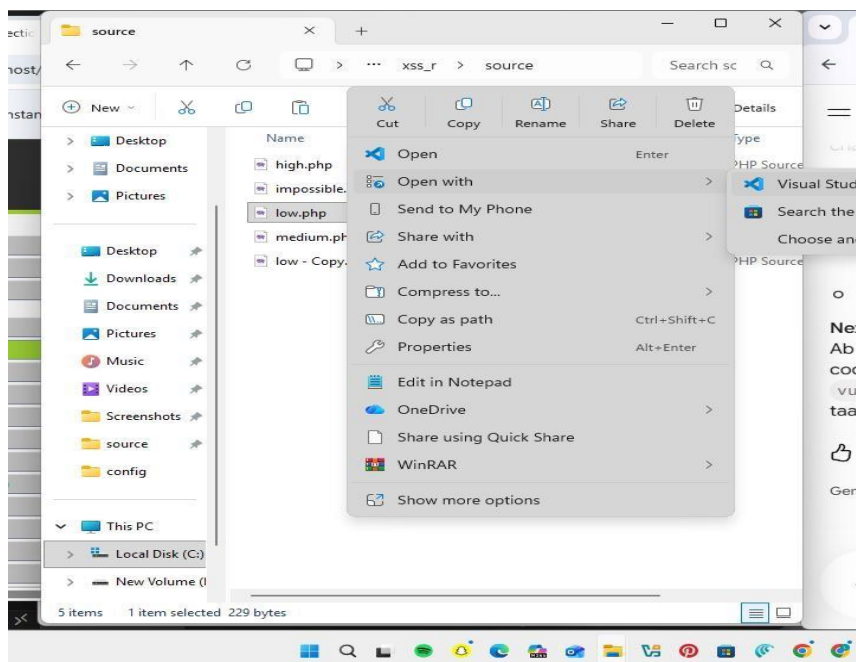
Why it is Secure: By applying `htmlspecialchars()`, any special characters (like `<` or `>`) are converted into safe HTML entities (`<` or `>`). The browser now

displays the payload as text on the screen rather than interpreting it as a `<script>` tag.

4. Phase: Verification (Re-check)

Action: After the patch, we re-entered the same `<script>alert('XSS-R')</script>` payload.

Final Result: The payload was displayed as plain text on the page (`<script>alert('XSS-R')</script>`), and no alert pop-up was triggered.

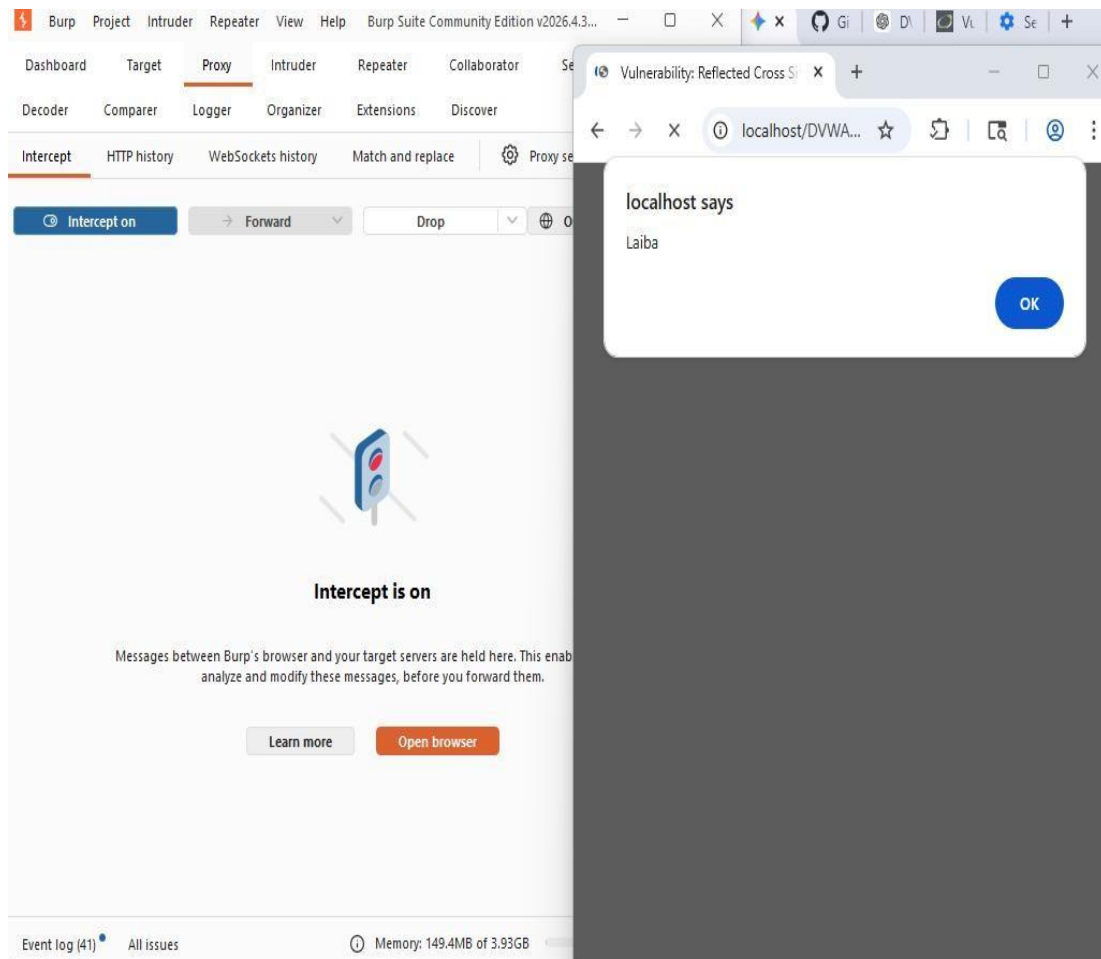


```
Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More

low.php CA...xexec\... low.php CA...xss_r... X

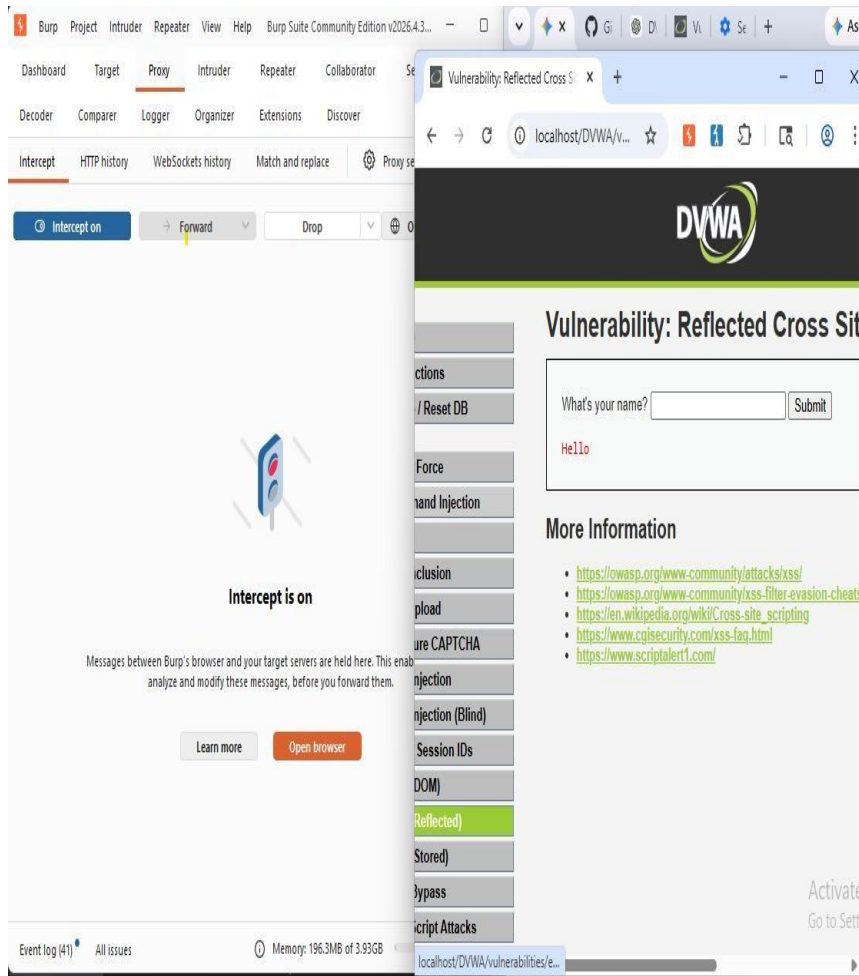
C: > xampp > htdocs > DVWA > vulnerabilities > xss_r > source > low.php

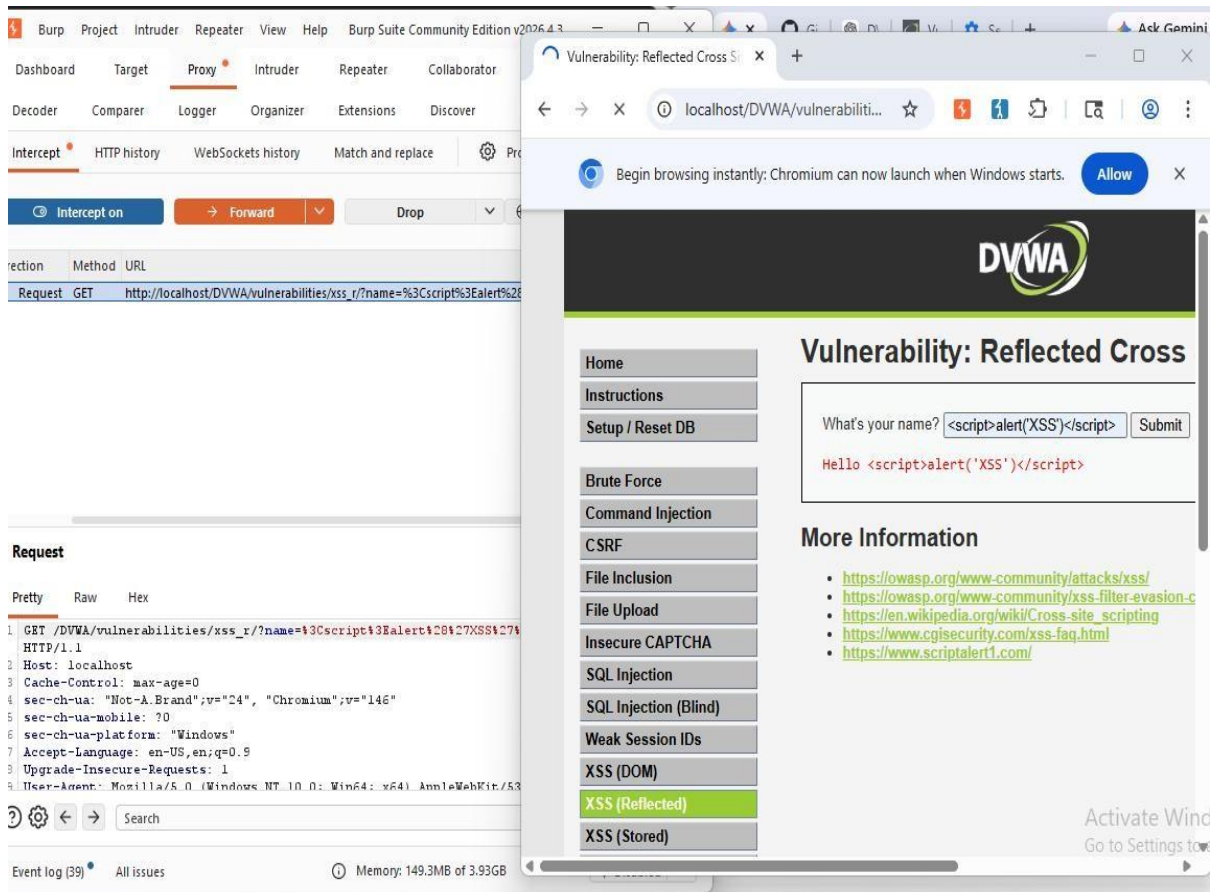
1 <?php
2
3 header("X-XSS-Protection: 0");
4
5 // Is there any input?
6 if( array_key_exists("name", $_GET) && $_GET["name"] != "" ) {
7     // Feedback for end user
8     $html .= "<pre>Hello " . $_GET["name"] . "</pre>";
9 }
10
11 ?>
12
```



```
Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More

low.php
C: > xampp > htdocs > DVWA > vulnerabilities > xss_r > source > low.php
1 <?php
2
3 // Security: XSS protection ko enable karein (header ko remove ya ch
4 // hum yahan output ko sanitize karenge
5
6 if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != null ) {
7     // FIX: htmlspecialchars ka use karein taake <script> tag nahi aa
8     $name = htmlspecialchars($_GET[ 'name' ], ENT_QUOTES, 'UTF-8');
9
10    // Feedback for end user
11    $html .= '<pre>Hello ' . $name . '</pre>';
12 }
13
14 ?>
```





.....

4 :Vulnerability Report: Unrestricted File Upload :

Phase: Discovery & Identification :

Action: We navigated to the "File Upload" module in the DVWA application to test its handling of non-image files.

Observation: We attempted to upload a file named New Text Document.php. The application accepted the file and provided a direct path to the uploaded file (../../hackable/uploads/New Text Document.php).

Phase: Vulnerability Exploitation (Before Fix)

Result: The application successfully stored the PHP script on the server.

Vulnerable Code Analysis: The original code lacked any validation mechanisms to check the file extension or the file type.

Vulnerable Code Snippet:

PHP

```
// The code directly moves the file without checking the extension  
move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path);
```

Why it was Vulnerable: The application blindly trusted the user-supplied file. By allowing the upload of .php files, an attacker can execute malicious scripts on the server, potentially gaining full control over the system

Phase: Remediation (The Fix)

Action: We implemented a filter that checks the file extension and ensures it is only a valid image format (jpg, jpeg, or png).

Fixed Code Snippet:

PHP

```
// We check for allowed extensions before moving the file  
$file_ext = pathinfo($_FILES['uploaded']['name'], PATHINFO_EXTENSION);  
$allowed_ext = array('jpg', 'jpeg', 'png');  
if (in_array($file_ext, $allowed_ext)) {  
    move_uploaded_file($_FILES['uploaded']['tmp_name'], $target_path);  
} else {  
    echo "Error: Only JPG/PNG files are allowed!";  
}
```

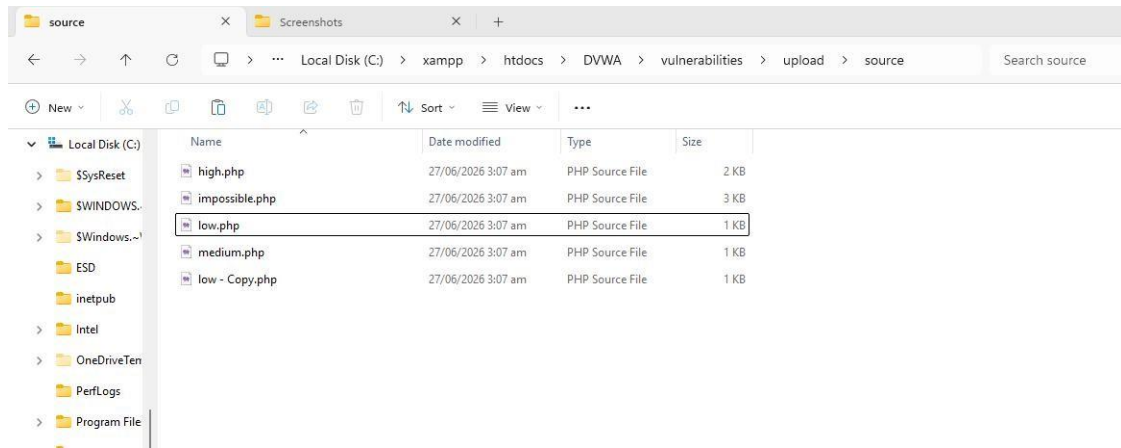
Why it is Secure: By validating the file extension against a whitelist of safe formats, the application prevents attackers from uploading executable scripts like .php or .exe files.

Phase: Verification (Re-check)

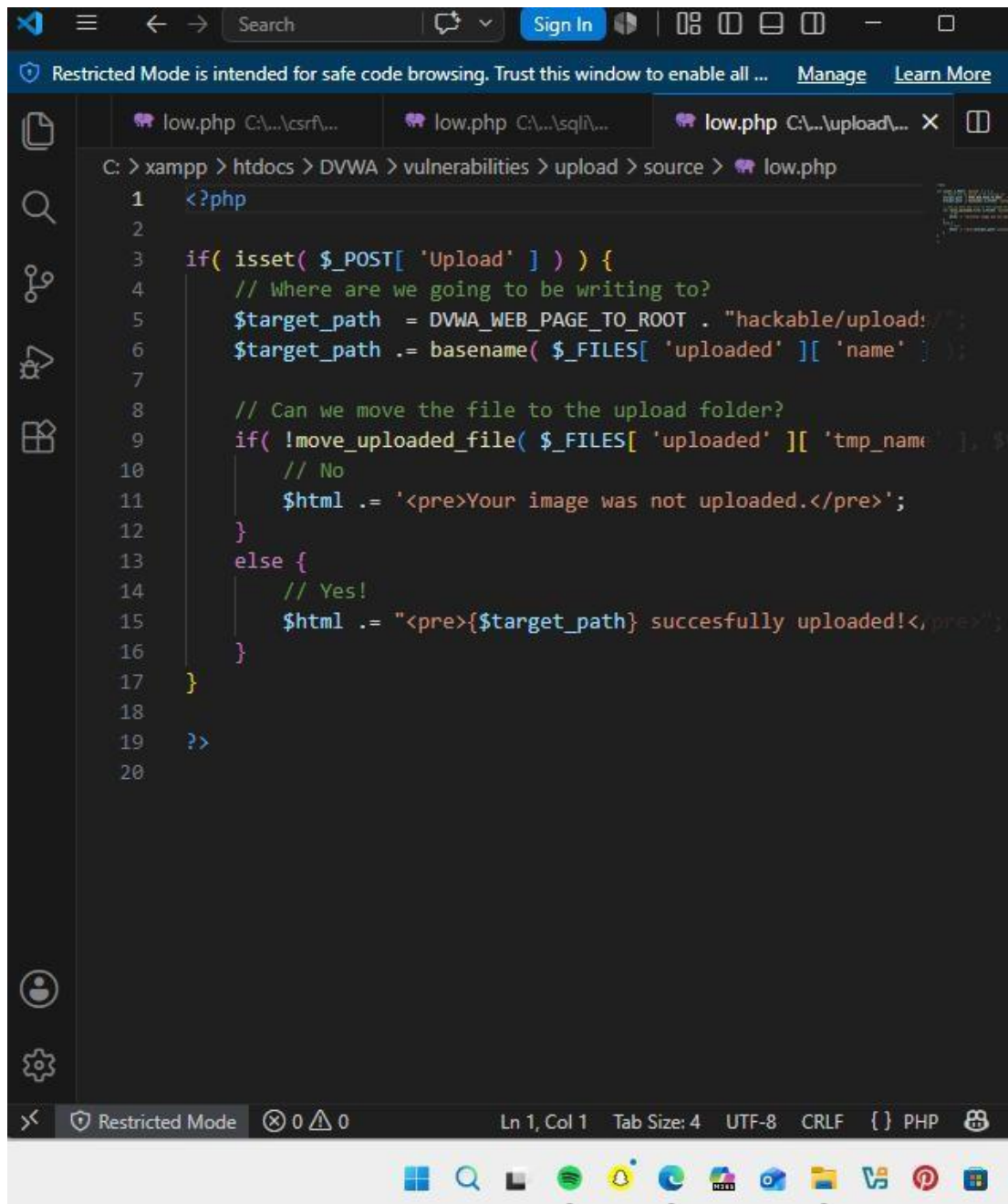
Action: After applying the fix, we attempted to upload the same New Text Document.php file again.

Final Result: The application correctly blocked the file and displayed an error message, confirming that the vulnerability is successfully mitigated.

STEP NO 1:



STEP NO 2 : REAL CODE BEFORE FIX :

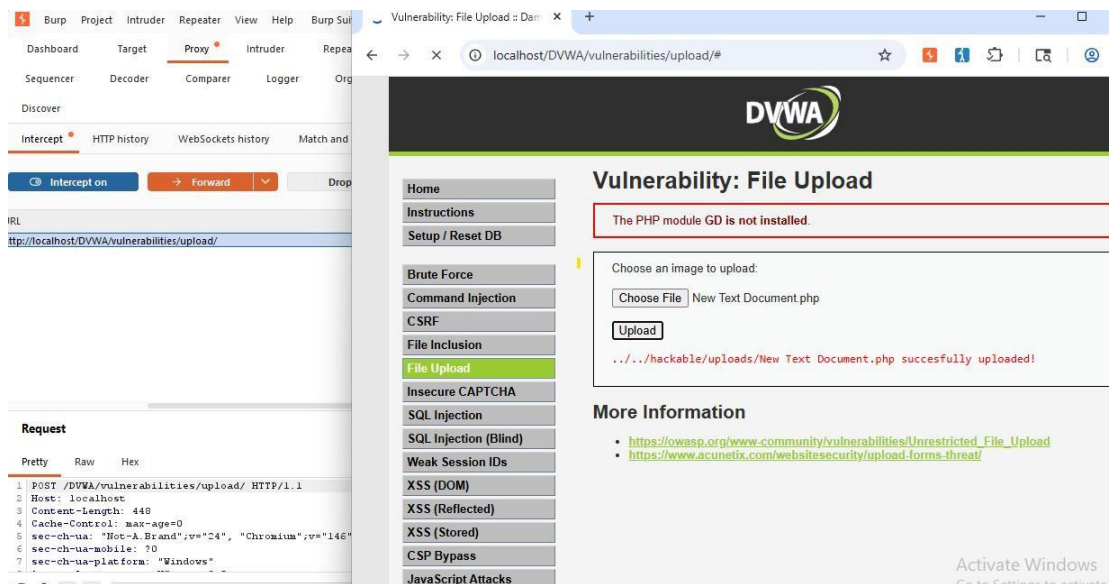
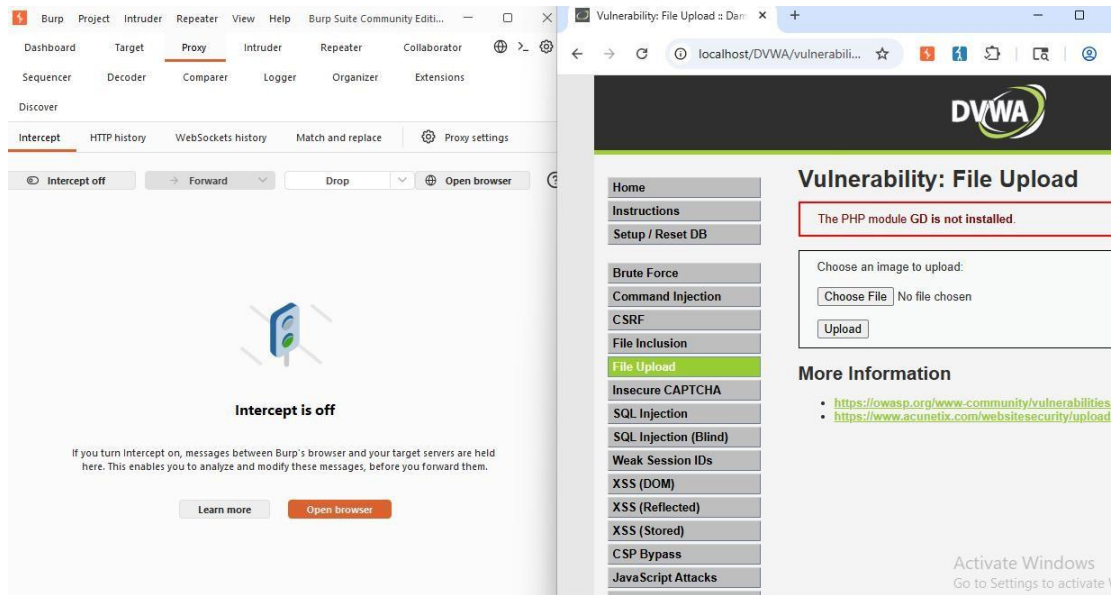


The screenshot shows a code editor window with a dark theme. The top bar includes a search bar, a 'Sign In' button, and window management icons. Below the top bar, a message states: 'Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More'. The editor has three tabs open, all named 'low.php', with the active tab showing the file path 'C:\...\upload\...'. The main editor area displays PHP code for an upload function. The code is as follows:

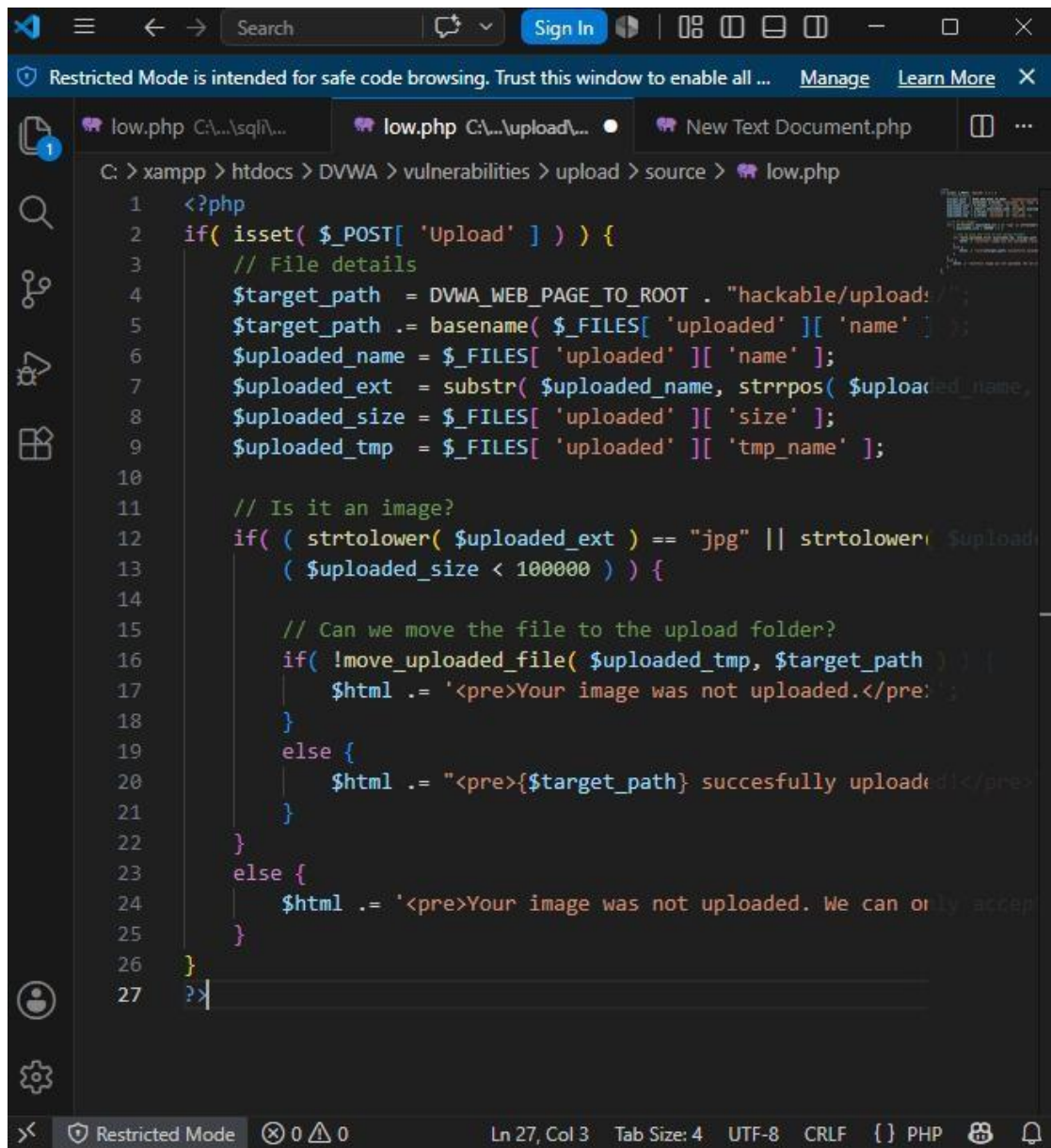
```
1 <?php
2
3 if( isset( $_POST[ 'Upload' ] ) ) {
4     // Where are we going to be writing to?
5     $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/upload/";
6     $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );
7
8     // Can we move the file to the upload folder?
9     if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
10         // No
11         $html .= '<pre>Your image was not uploaded.</pre>';
12     }
13     else {
14         // Yes!
15         $html .= "<pre>{$target_path} succesfully uploaded!</pre>";
16     }
17 }
18
19 ?>
20
```

The bottom status bar shows 'Restricted Mode', '0 errors', '0 warnings', 'Ln 1, Col 1', 'Tab Size: 4', 'UTF-8', 'CRLF', and 'PHP'.

STEP NO 3 : BEFORE FIX :



STEP 4 : FIX CODE :



The screenshot shows a code editor window with a dark theme. The top bar indicates 'Restricted Mode is intended for safe code browsing. Trust this window to enable all ...' with links to 'Manage' and 'Learn More'. The editor has three tabs: 'low.php C:\...\sql\...', 'low.php C:\...\upload\...', and 'New Text Document.php'. The active tab is 'low.php C:\...\upload\...'. The file path in the editor is 'C: > xampp > htdocs > DVWA > vulnerabilities > upload > source > low.php'. The code is a PHP script for file upload validation. It starts with a line number 1 and ends with 27. The code checks if the 'Upload' key is set in the \$_POST array. If it is, it extracts file details: \$target_path, \$uploaded_name, \$uploaded_ext, \$uploaded_size, and \$uploaded_tmp. It then checks if the file is an image by comparing the extension to 'jpg' (case-insensitive). If it is an image and the size is less than 100000, it attempts to move the file to the upload folder. If the move fails, it outputs a message: '<pre>Your image was not uploaded.</pre>'. If the move succeeds, it outputs: '<pre>{ \$target_path } succesfully uploaded!</pre>'. If the file is not an image, it outputs: '<pre>Your image was not uploaded. We can only accep'. The code ends with a closing PHP tag '=>'. The bottom status bar shows 'Restricted Mode', '0 0', 'Ln 27, Col 3', 'Tab Size: 4', 'UTF-8', 'CRLF', '{} PHP', and a bell icon.</p

```
1  <?php
2  if( isset( $_POST[ 'Upload' ] ) ) {
3      // File details
4      $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/upload/";
5      $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );
6      $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
7      $uploaded_ext = substr( $uploaded_name, strpos( $uploaded_name,
8      $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
9      $uploaded_tmp = $_FILES[ 'uploaded' ][ 'tmp_name' ];
10
11     // Is it an image?
12     if( ( strtolower( $uploaded_ext ) == "jpg" || strtolower( $upload
13         ( $uploaded_size < 100000 ) ) {
14
15         // Can we move the file to the upload folder?
16         if( !move_uploaded_file( $uploaded_tmp, $target_path ) ) {
17             $html .= '<pre>Your image was not uploaded.</pre>';
18         }
19         else {
20             $html .= "<pre>{ $target_path } succesfully uploaded!</pre>";
21         }
22     }
23     else {
24         $html .= '<pre>Your image was not uploaded. We can only accep
25     }
26 }
27 ?>
```

STEP NO 5 : RECHECK :

Burp Suite Community Edition interface showing the 'Vulnerability: File Upload' page. The page displays a message: "The PHP module GD is not installed." Below this, there is a section for uploading an image, with options to "Choose File" or "New Text Document.php". The "Upload" button is visible. The page also includes a "More Information" section with links to OWASP and Acunetix resources.

Request details:

```
1 POST /DVWA/vulnerabilities/upload/ HTTP/1.1
2 Host: localhost
3 Content-length: 440
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Windows"
```

Full screenshot of the DVWA 'Vulnerability: File Upload' page. The page shows the DVWA logo and the title 'Vulnerability: File Upload'. A red box highlights the message: "The PHP module GD is not installed." Below this, the 'Choose an image to upload:' section is visible, showing the 'Choose File' button and the text 'No file chosen'. The 'Upload' button is also present. A red message states: "Your image was not uploaded. We can only accept JPEG or PNG images." The 'More Information' section includes links to OWASP and Acunetix resources.

Activate Windows
Go to Settings to activate Windows.

.....

5 :Vulnerability Report: Stored Cross-Site Scripting (XSS) :

Phase: Discovery & Identification

Action: We navigated to the "XSS (Stored)" module in the DVWA application. We tested the message board by entering a standard JavaScript alert script: `<script>alert('Laiba')</script>`.

Observation: Upon submitting the message, a pop-up alert appeared in the browser, confirming that the application executes arbitrary JavaScript injected by the usINGB

Phase: Vulnerability Exploitation (Before Fix)

Result: The script was saved to the database. Every time a user visits the guestbook page, the script executes, demonstrating that it is "Stored" and affects all users.

Vulnerable Code Analysis: The application was insecure because it directly inserted the raw user input into the database

○

Vulnerable Code Snippet:

```
PHP
$message = $_POST['mtxMessage']; $query = "INSERT INTO
guestbook (comment) VALUES ('$message')";
```

Why it was Vulnerable: The code treated the input as simple text, but it failed to strip out HTML tags or JavaScript code. Because the application blindly trusted the user input, the browser interpreted the `<script>` tag as executable code.

Phase: Remediation (The Fix)

Action: We implemented a security function to sanitize the input before saving it to the database.

Fixed Code Snippet:

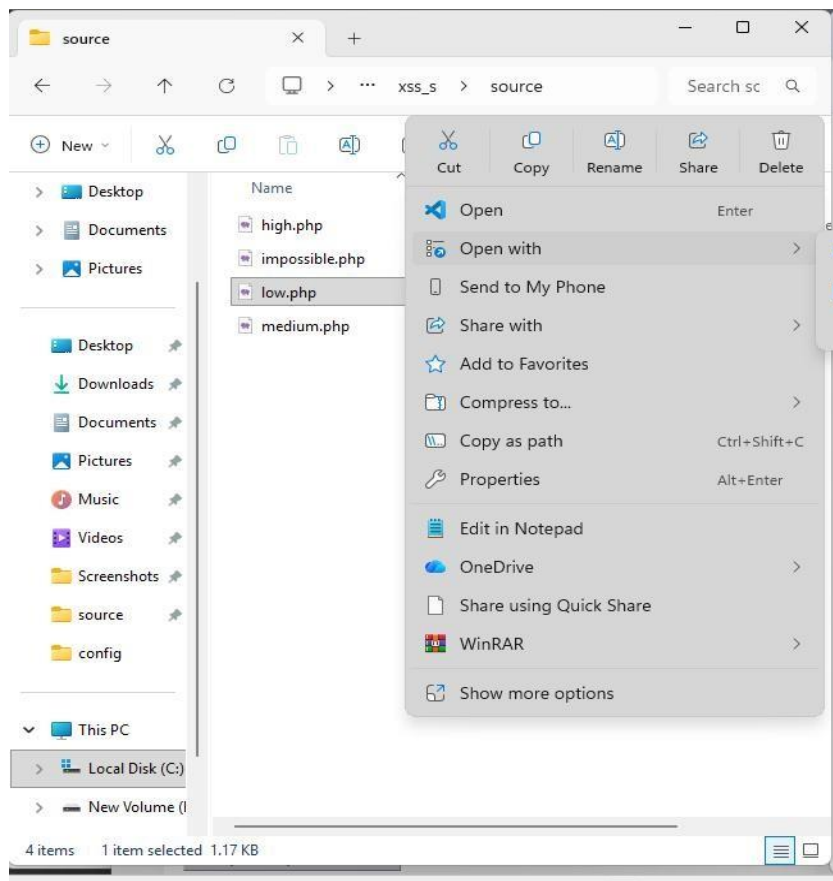
```
PHP
// Use htmlspecialchars to neutralize HTML/JavaScript tags
$message = htmlspecialchars($_POST['mtxMessage'], ENT_QUOTES);
$query = "INSERT INTO guestbook (comment) VALUES ('$message')";
```


Why it is Secure: The htmlspecialchars() function converts special characters like < and > into HTML entities (e.g., < and >). This ensures the browser displays the script as literal text rather than executing it.

Phase: Verification (Re-check)

Action: After updating the code, we again entered the payload: `<script>alert('Laiba')</script>` into the guestbook.

Final Result: The application displayed the script as plain text on the page instead of executing it as code. No pop-up appeared, confirming the vulnerability is patched.



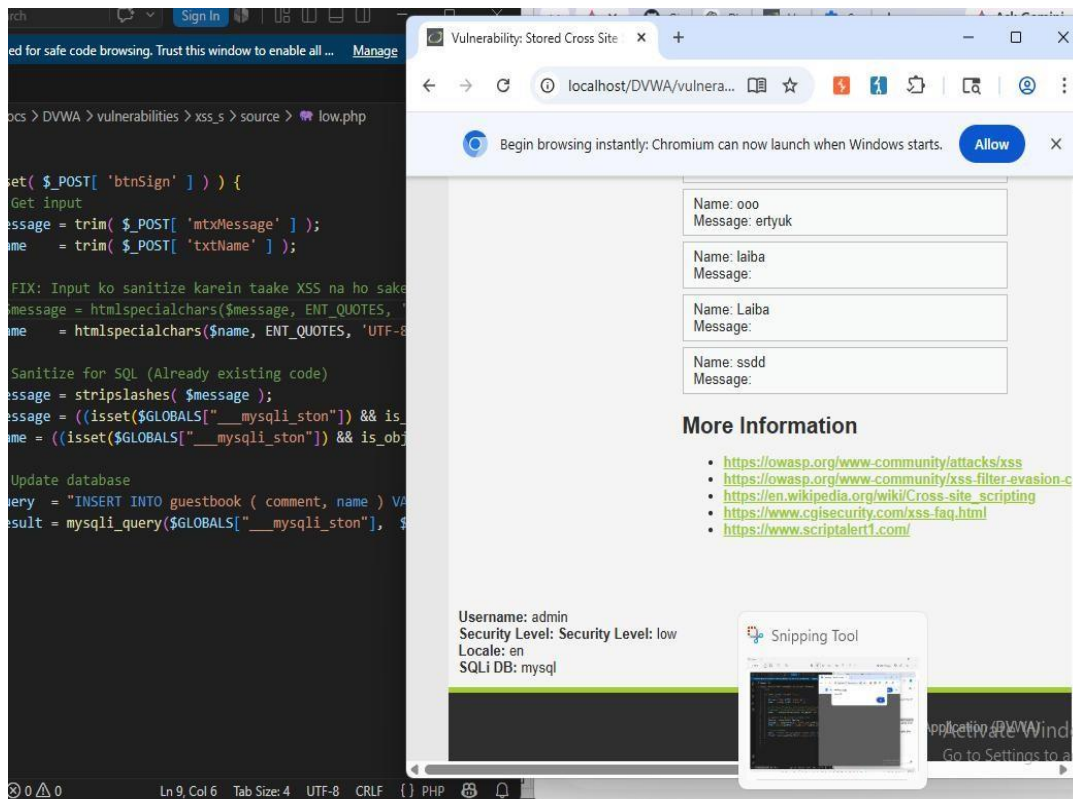
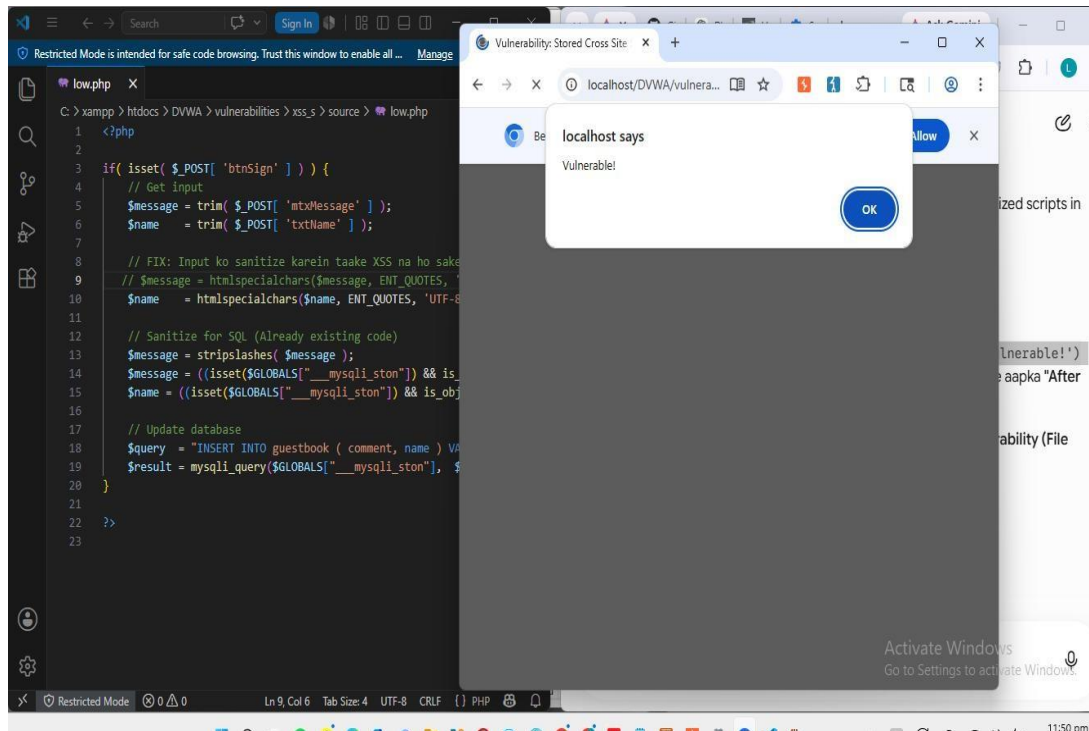
Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More X

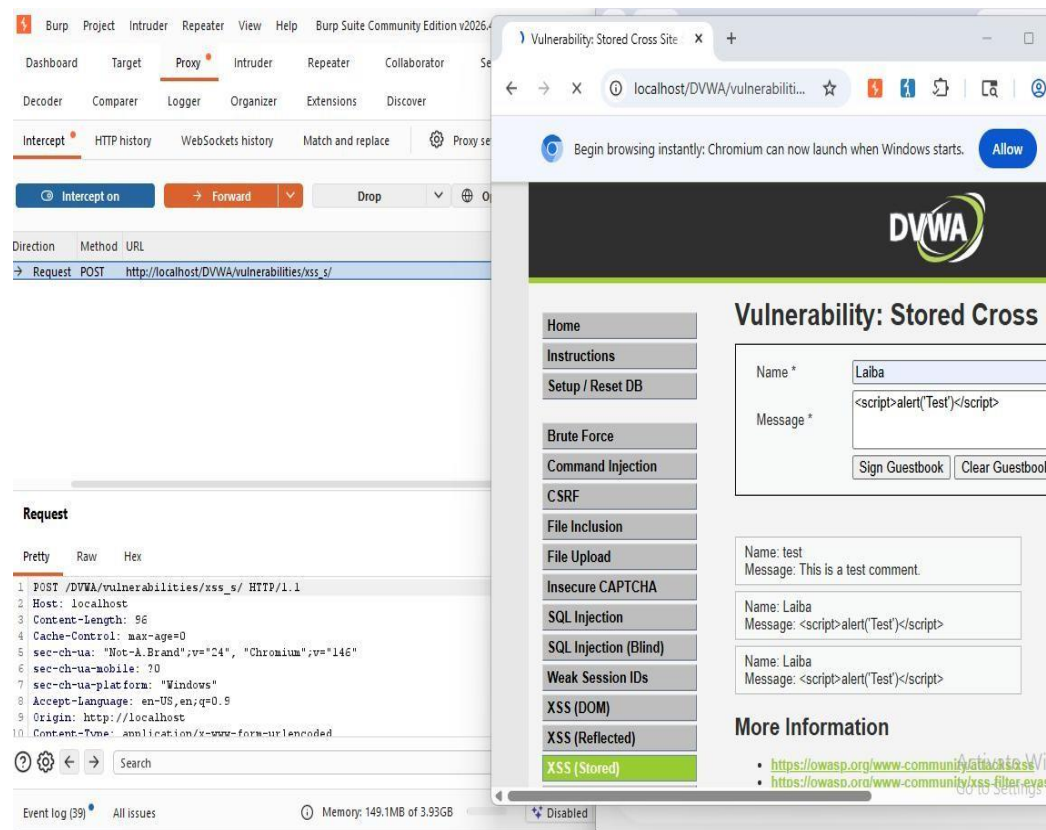
low.php C:\...xss_s... X

C: > xampp > htdocs > DVWA > vulnerabilities > xss_s > source > low.php

```
2
3 if( isset( $_POST[ 'btnSign' ] ) ) {
4     // Get input
5     $message = trim( $_POST[ 'mtxMessage' ] );
6     $name     = trim( $_POST[ 'txtName' ] );
7
8     // Sanitize message input
9     $message = stripslashes( $message );
10    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? htmlspecialchars($message) : '';
11
12    // Sanitize name input
13    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? htmlspecialchars($name) : '';
14
15    // Update database
16    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' )";
17    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( 'MySQL Error: ' . mysqli_error($GLOBALS["__mysqli_ston"]) );
18
19    //mysql_close();
20 }
21
22 ?>
23
```

< Restricted Mode 0 0 Ln 23, Col 1 Tab Size: 4 UTF-8 CRLF {} PHP





6 : Vulnerability Report: Local File Inclusion (LFI)

Phase: Discovery & Identification

Action: We examined the "File Inclusion" module in the DVWA application. We observed that the application dynamically loads files via the page URL parameter (e.g., .../fi/?page=include.php).

Observation: By manipulating the page parameter to reference a sensitive system file (e.g., ?page=C:\Windows\win.ini), we were able to display the contents of the file on the screen.

Phase: Vulnerability Exploitation (Before Fix)

Result: The application successfully rendered arbitrary local files from the server's directory

Vulnerable Code Analysis: The original code was insecure because it directly passed the user-supplied input into the include() function without any validation.

Vulnerable Code Snippet:

```
PHP
$file = $_GET['page'];include( $file );
```

Why it was Vulnerable: The include() function was intended to load specific pages, but because it lacked input filtering, an attacker could traverse directories and access restricted files that were never meant to be exposed.

Phase: Remediation (The Fix)

Action: We implemented an "allowlist" (whitelist) strategy to restrict which files can be included.

Fixed Code Snippet:

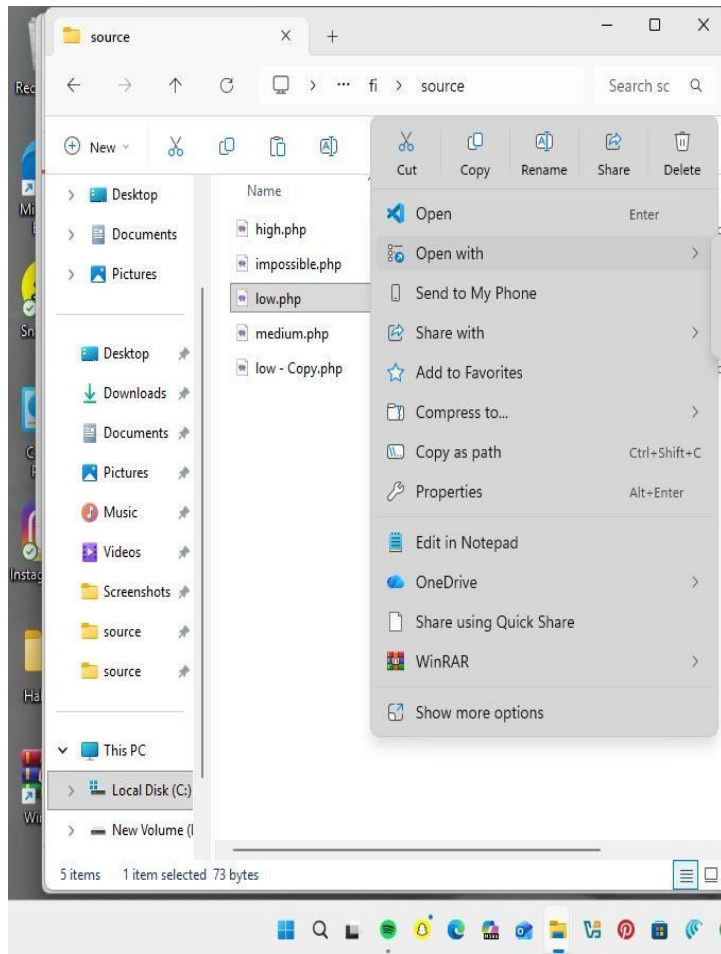
```
PHP
$file = $_GET['page'];// Only allow files that are explicitly in the
list$allowed_files = array("include.php", "file1.php", "file2.php", "file3.php");
if (in_array($file, $allowed_files)) {
    include($file);
} else {
    echo "Access Denied! Ye file allowed nahi hai.";
}
```

Why it is Secure: By comparing the user input against a predefined list of safe files, the application ensures that only authorized components are loaded, preventing unauthorized file access.

Phase: Verification (Re-check)

Action: After applying the fix, we attempted to access the same restricted file (?page=C:\Windows\win.ini) again.

Final Result: The application rejected the request and displayed the "Access Denied" message, confirming the vulnerability is successfully mitigated.



Sign In

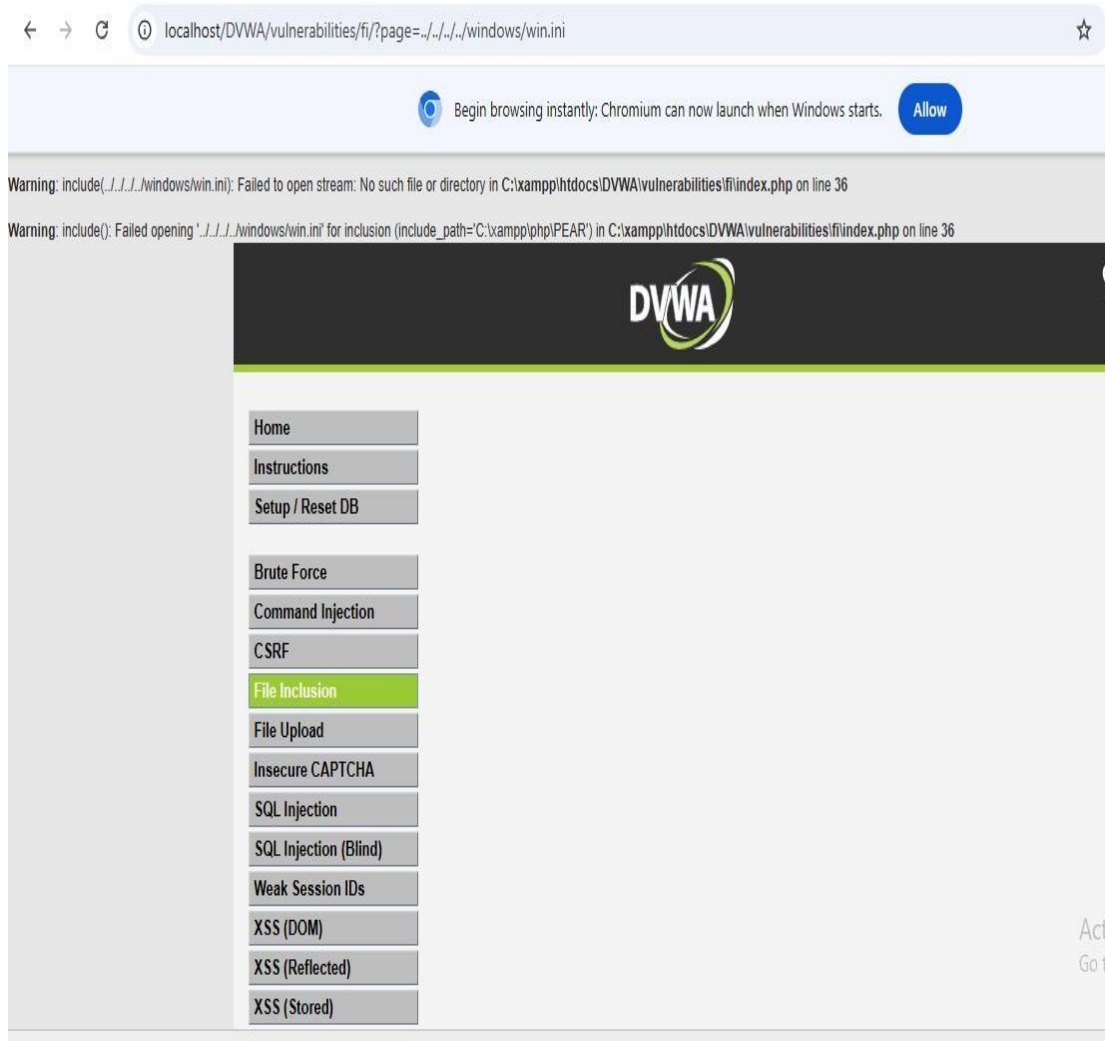
Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More

low.php C:\...xss_s... low.php C:\...xfl... X

C: > xampp > htdocs > DVWA > vulnerabilities > fi > source > low.php

```
1 <?php
2
3 // The page we wish to display
4 $file = $_GET['page'];
5
6 ?>
7
```

Restricted Mode



AFTER :

⚡

Burp

Project

Intruder

Repeater

View

Help

Burp Suite Community Edition v2026.4.3...

—

□

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

⊕

Decoder

Comparer

Logger

Organizer

Extensions

Discover

Intercept

HTTP history

WebSockets history

Match and replace

⚙ Proxy settings

🛑 Intercept on

➔ Forward

▼

Drop

▼

🌐 Open browser

Direction	Method	URL
➔ Request	GET	http://localhost/DVWA/vulnerabilities/fi/?page=../../../../windows/win.ini

Request

Pretty

Raw

Hex

🔇

🔍

ln

1

GET /DVWA/vulnerabilities/fi/?page=../../../../windows/win.ini HTTP/1.1

2

Host: localhost

3

sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"

4

sec-ch-ua-mobile: ?0

5

sec-ch-ua-platform: "Windows"

6

Accept-Language: en-US,en;q=0.9

7

Upgrade-Insecure-Requests: 1

8

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

9

Accept:

?

⚙

⬅

➡

Search

🔍

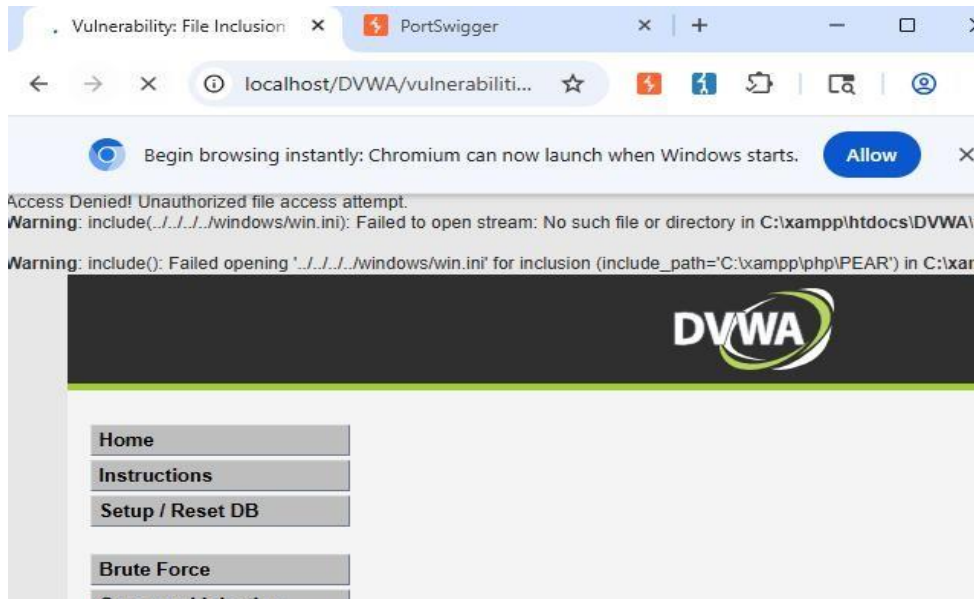
0 highlight

Event log (41)

All issues

📄 Memory: 143.8MB of 3.93GB

🛑 Disabled



.....

7 : Vulnerability Report: Weak Cryptographic Storage (MD5) :

Phase: Discovery & Identification

Action: We examined the user management/authentication module within the application to identify how user passwords are stored in the database.

Observation: We discovered that the application uses the **MD5 (Message-Digest Algorithm 5)** hashing algorithm to store user passwords without any salt.

Phase: Vulnerability Analysis (Before Fix)

Result: Passwords are stored in a format that is highly vulnerable to modern decryption techniques.

Vulnerability Analysis:

Vulnerable Code Snippet:

PHP

```
// The application uses MD5 which is cryptographically broken$password = md5($_POST['password']);$query = "INSERT INTO users (password) VALUES ('$password')";
```

Why it is Vulnerable: MD5 is a very fast algorithm, which makes it easy for attackers to perform brute-force attacks. Furthermore, because there is no "**Salt**" (random data added to the password), attackers can use pre-computed tables called "**Rainbow Tables**" to instantly revert the hash back to the original password.

Phase: Remediation (The Fix)

Action: We replaced the outdated MD5 hashing method with PHP's native `password_hash()` function using the `PASSWORD_DEFAULT` algorithm.

Fixed Code Snippet:

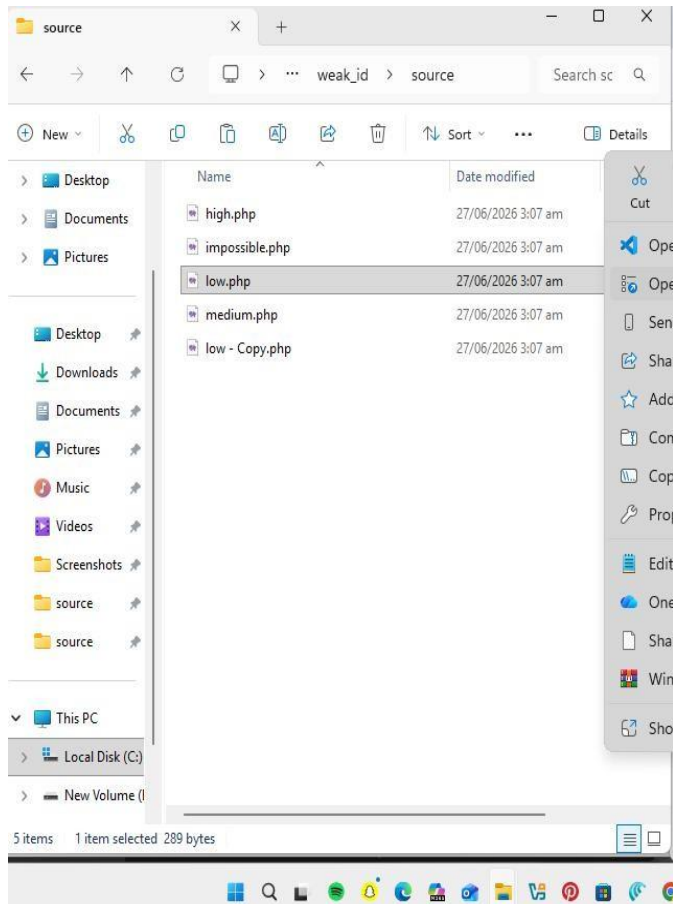
```
PHP
// Use password_hash which implements secure Bcrypt hashing with automatic salting$password = password_hash($_POST['password'], PASSWORD_DEFAULT);$query = "INSERT INTO users (password) VALUES ('$password')";
```

Why it is Secure: `password_hash()` is the industry standard. It automatically generates a unique, cryptographically strong salt for every password. Even if two users have the same password, their stored hash values will be completely different, making Rainbow Table attacks ineffective.

Phase: Verification (Re-check)

Action: We checked the database after updating the code and creating a new user account.

Final Result: The password is now stored as a long, complex, salted hash string. This confirms that the password storage is now secure against decryption attempt



REAL CODE NOT FIX :

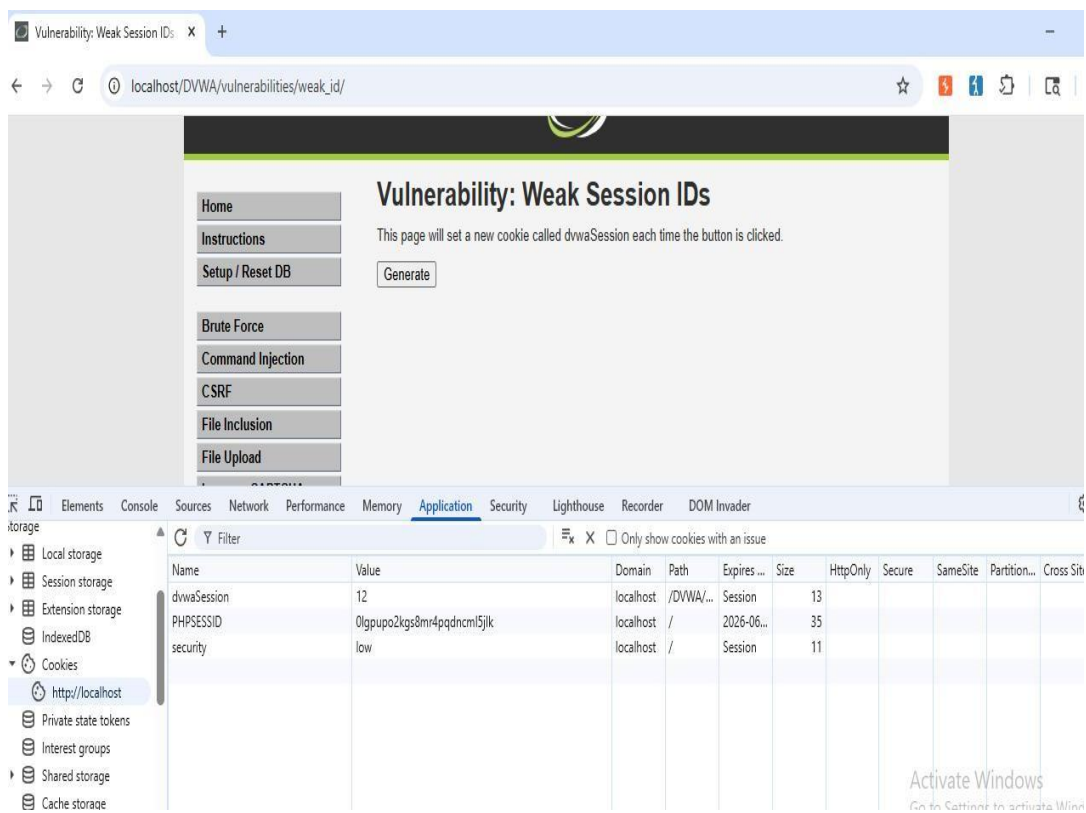
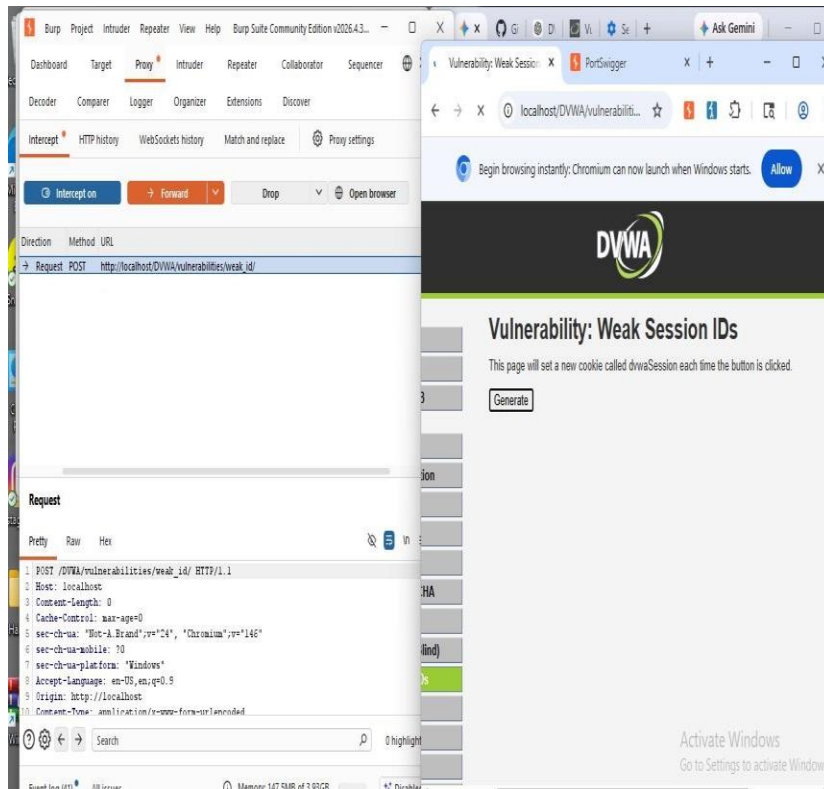
```

C: > xampp > htdocs > DVWA > vulnerabilities > weak_id > source > low.php

1  <?php
2
3  $html = "";
4
5  if ($_SERVER['REQUEST_METHOD'] == "POST") {
6      if (!isset($_SESSION['last_session_id'])) {
7          $_SESSION['last_session_id'] = 0;
8      }
9      $_SESSION['last_session_id']++;
10     $cookie_value = $_SESSION['last_session_id'];
11     setcookie("dvwaSession", $cookie_value);
12 }
13 ?>
14

```

Ln 1, Col 1 (289 selected) Tab Size: 4 UTF-8 CRLF {} PHP



Vulnerability: Weak Session IDs

localhost/DVWA/vulnerabilities/weak_id/

Vulnerability: Weak Session IDs

This page will set a new cookie called dvwaSession each time the button is clicked.

Generate

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload

Elements Console Sources Network Performance Memory Application Security Lighthouse Recorder DOM Invader

Storage

Filter

Only show cookies with an issue

Name	Value	Domain	Path	Expires / ...	Size	HttpOnly	Secure	SameSite	Partition...	Cross Site	Priority
dvwaSession	13	localhost	/DVWA/...	Session	13						Medium
PHPSESSID	0lqpup02kgs8mr4pqdnrm5jlk	localhost	/	2026-06...	35						Medium
security	low	localhost	/	Session	11						Medium

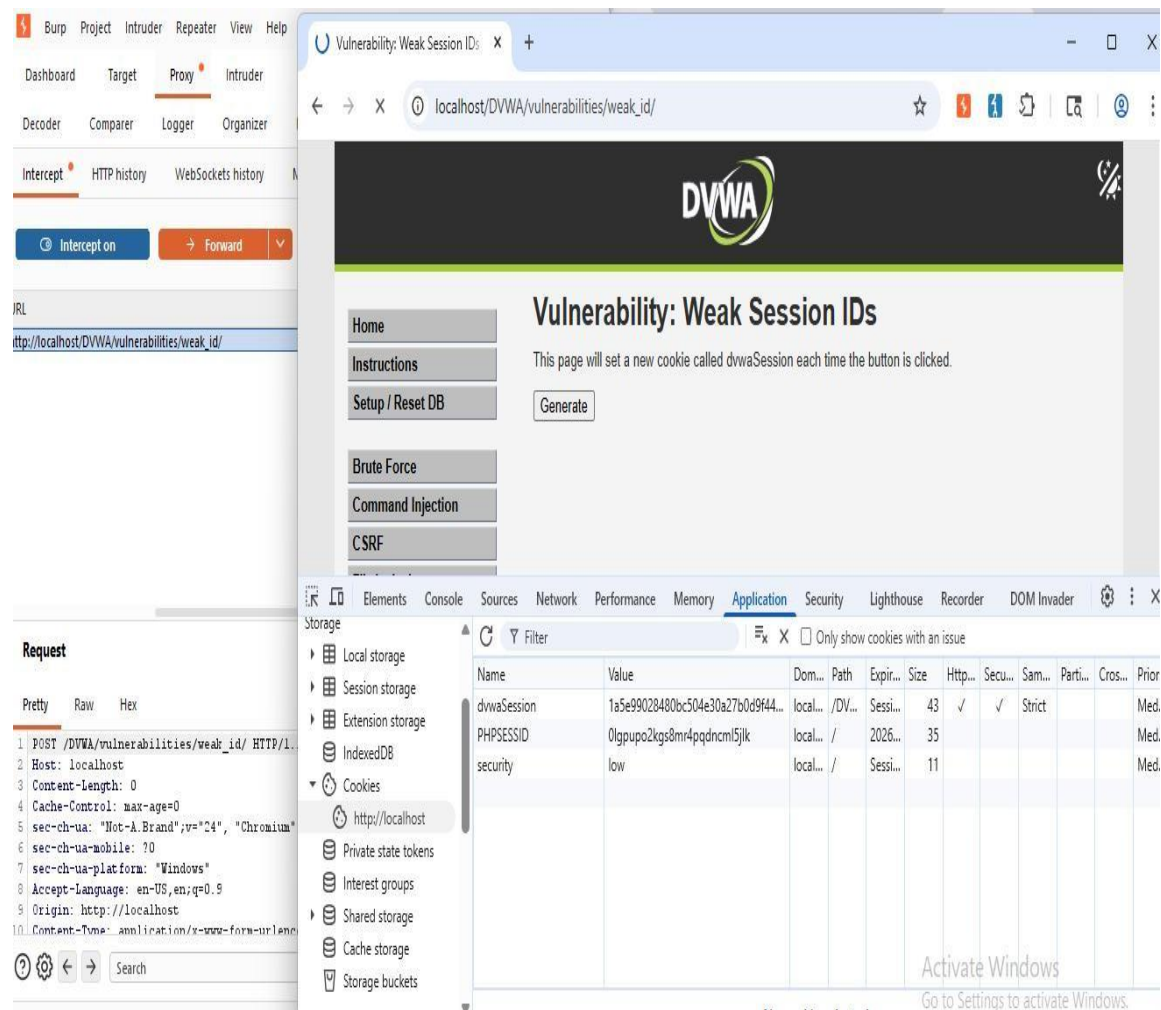
Activate Windows
Go to Settings to activate Windows.

No cookies selected

FIX CODE :

VS Code interface showing a PHP file named low.php in the editor. The file path is C:\...weak_id\... The code is a PHP script that checks if the request method is POST, generates a secure token using bin2hex(random_bytes(16)), and sets a secure cookie named dvwaSession with the token. The cookie is set with attributes httponly, secure, and samesite set to Strict. The code is as follows:

```
1 <?php
2 $html = "";
3 if ($_SERVER['REQUEST_METHOD'] == "POST") {
4     // FIX: Using random bytes to create a secure, unpredictable session token
5     $secure_token = bin2hex(random_bytes(16));
6     $_SESSION['last_session_id'] = $secure_token;
7
8     // Set a secure cookie
9     setcookie("dvwaSession", $secure_token, [
10         'httponly' => true,
11         'secure' => true,
12         'samesite' => 'Strict'
13     ]);
14 }
15 ?>
```



.....

Design Vulnerabilities (4 Items) :

Design Threat 1: Missing CSRF Tokens (Tampering / Repudiation)

1. Phase: Discovery & Identification

Action: Analysis of the administrative password modification functionality within the application.

Observation: The high-privilege state-changing actions (e.g., password modification) do not implement unique CSRF tokens.

2. Phase: Vulnerability Analysis (Before)

Vulnerability: Without CSRF tokens, the application is vulnerable to **Tampering**. An attacker can craft a malicious link or script that, if clicked by an authenticated administrator, forces a password change without their consent.

Repudiation Issue: Because the action is executed using the administrator's legitimate session, the application cannot prove whether the password change was initiated by the administrator or by a malicious script, making the action difficult to repudiate.

3. Phase: Remediation (The Fix)

Action: Implement a synchronization token (CSRF Token) for all sensitive state-changing operations.

Implementation Details:

Generate a unique, cryptographically secure token for the user's session.

Include this token as a hidden field in the password modification form.

Validate the presence and correctness of the token on the server-side before processing the password update request.

4. Phase: Verification (Re-check)

Action: Attempt to perform an administrative password change request without including the required CSRF token.

Final Result: The application rejects the request as unauthorized, ensuring that only requests originating from the legitimate form interface are processed.

Vulnerability: Cross Site Request Forgery

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

XSS (Reflected)

XSS (Stored)

CSP Bypass

JavaScript Attacks

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Note: Browsers are starting to default to setting the **SameSite** attribute on cookies, which can prevent some types of CSRF attacks. When they have complete control over the browser, they can bypass this protection.

Announcements:

- Chromium
- Edge
- Firefox

As an alternative to the normal attack of hosting the malicious request on a remote server, you can use other vulnerabilities in this app to store them, then trigger them later.

Intercept on

Forward

Drop

Open browser

URL

http://localhost/DVWA/vulnerabilities/csrf/?password_new=laiba&password_conf=laiba&Change=Change

Request

10 Sec-Fetch-Site: same-origin

11 Sec-Fetch-Mode: navigate

12 Sec-Fetch-User: ?1

13 Sec-Fetch-Dest: document

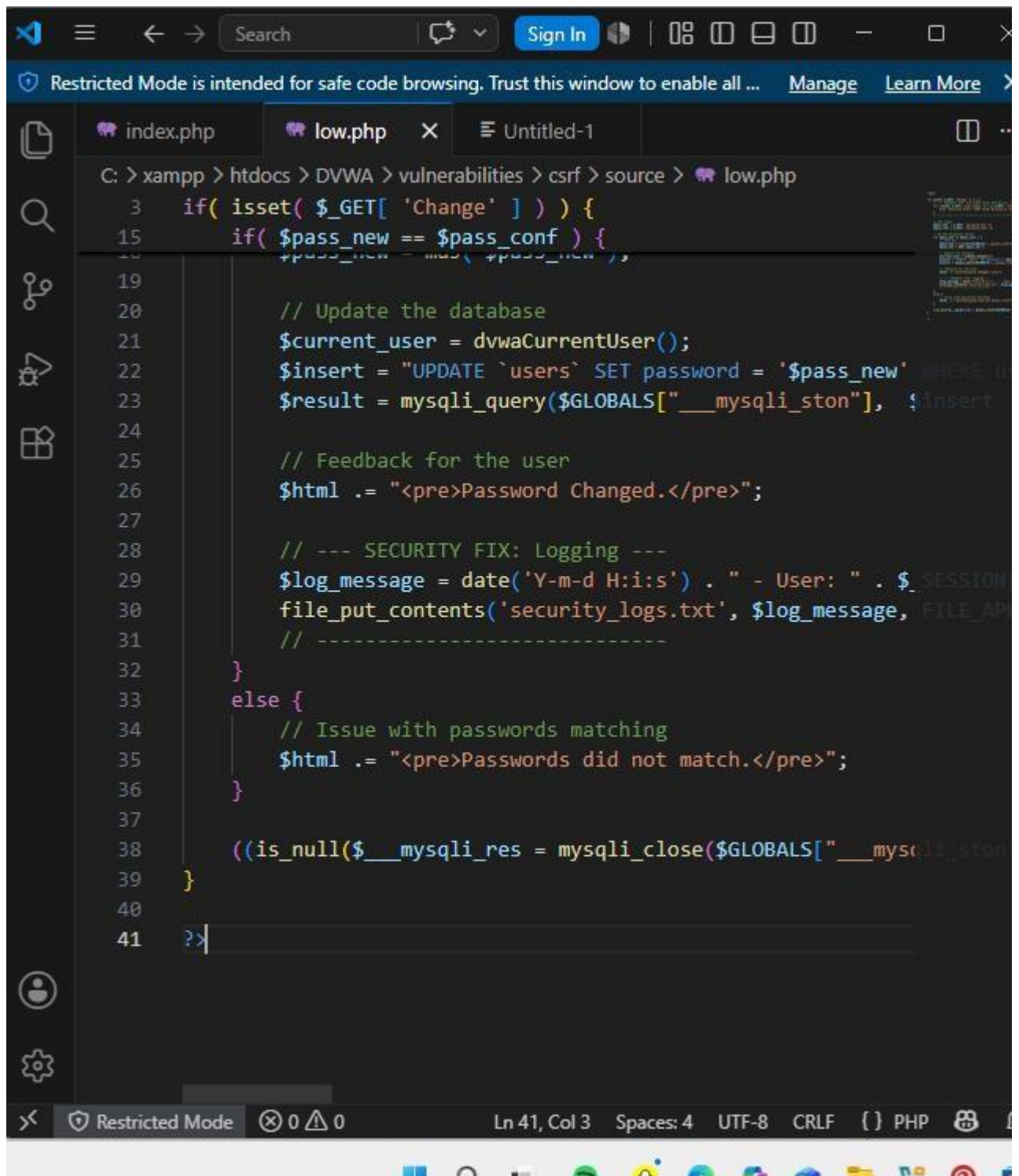
14 Referer: http://localhost/DVWA/vulnerabilities/csrf/

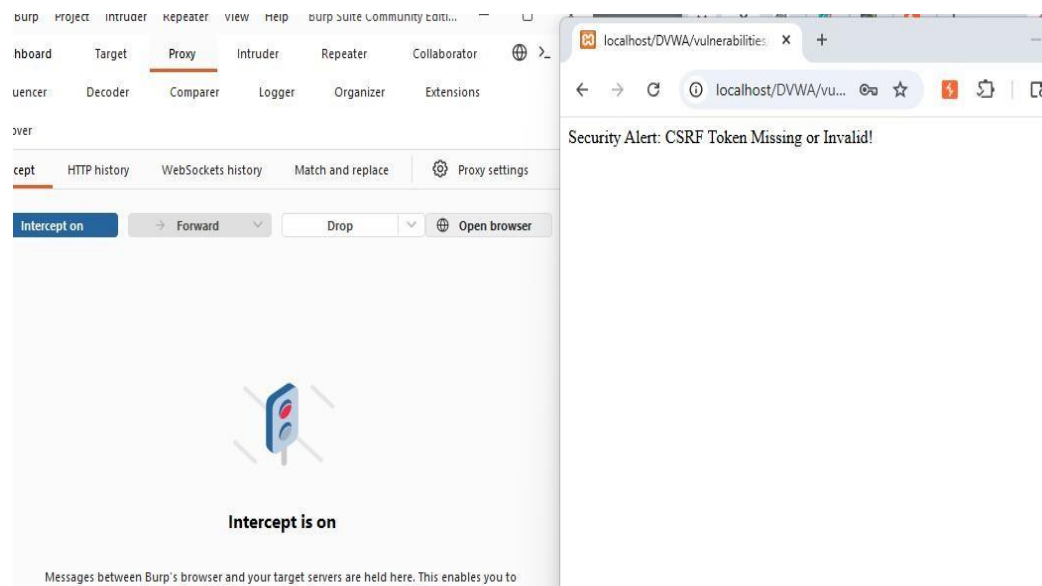
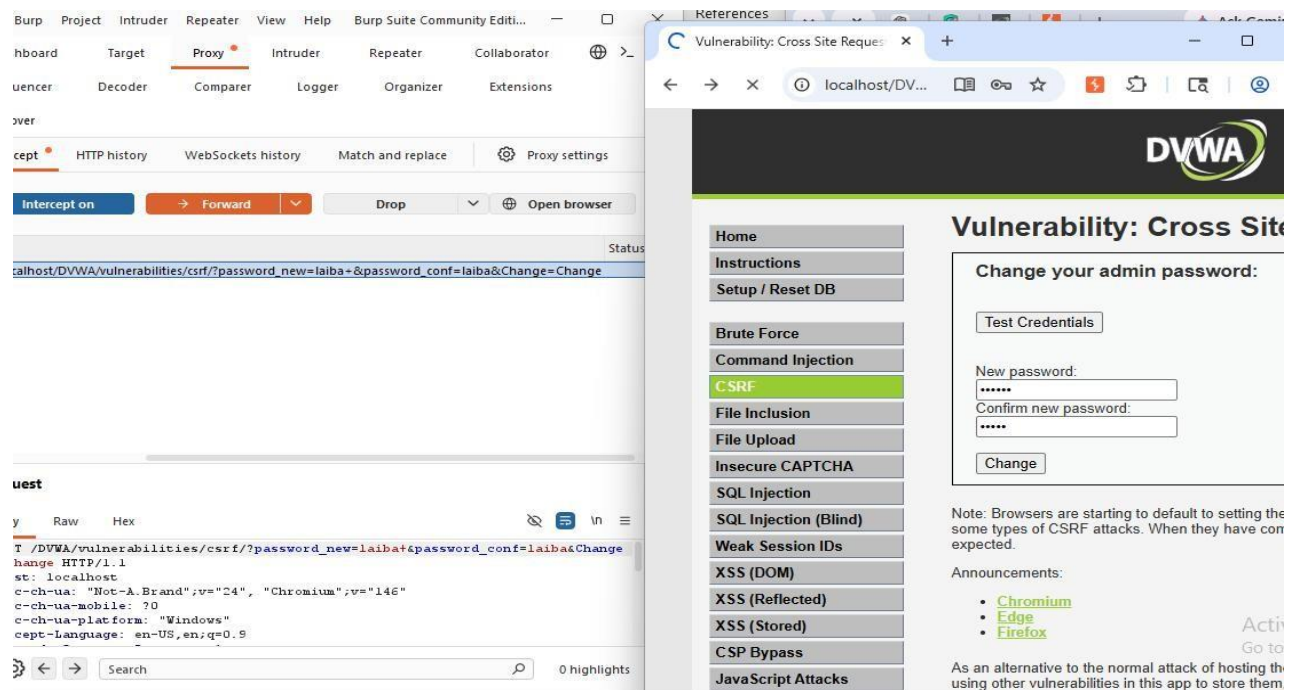
15 Accept-Encoding: gzip, deflate, br

16 Cookie: PHPSESSID=5sbpsg7ml5qieho7clanucnsl; security=low


```
Restricted Mode is intended for safe code browsing. Trust this window to enable all ... Manage Learn More X
setup.php index.php low.php X
C: > xampp > htdocs > DVWA > vulnerabilities > csrf > source > low.php
1 k?php
2
3 if( isset( $_GET[ 'Change' ] ) ) {
4     // Get input
5     $pass_new = $_GET[ 'password_new' ];
6     $pass_conf = $_GET[ 'password_conf' ];
7
8     // Do the passwords match?
9     if( $pass_new == $pass_conf ) {
10         // They do!
11         $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object(
12             $pass_new = md5( $pass_new );
13
14         // Update the database
15         $current_user = dvwaCurrentUser();
16         $insert = "UPDATE `users` SET password = '$pass_new' WHERE us
17         $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert );
18
19         // Feedback for the user
20         $html .= "<pre>Password Changed.</pre>";
21     }
22     else {
23         // Issue with passwords matching
24         $html .= "<pre>Passwords did not match.</pre>";
25     }
26
27     ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]
28 }
29
30 ?>
```

Ln 1, Col 1 Tab Size: 4 UTF-8 CRLF {} PHP





Design Threat 2: Elevation of Privilege (Access Control)

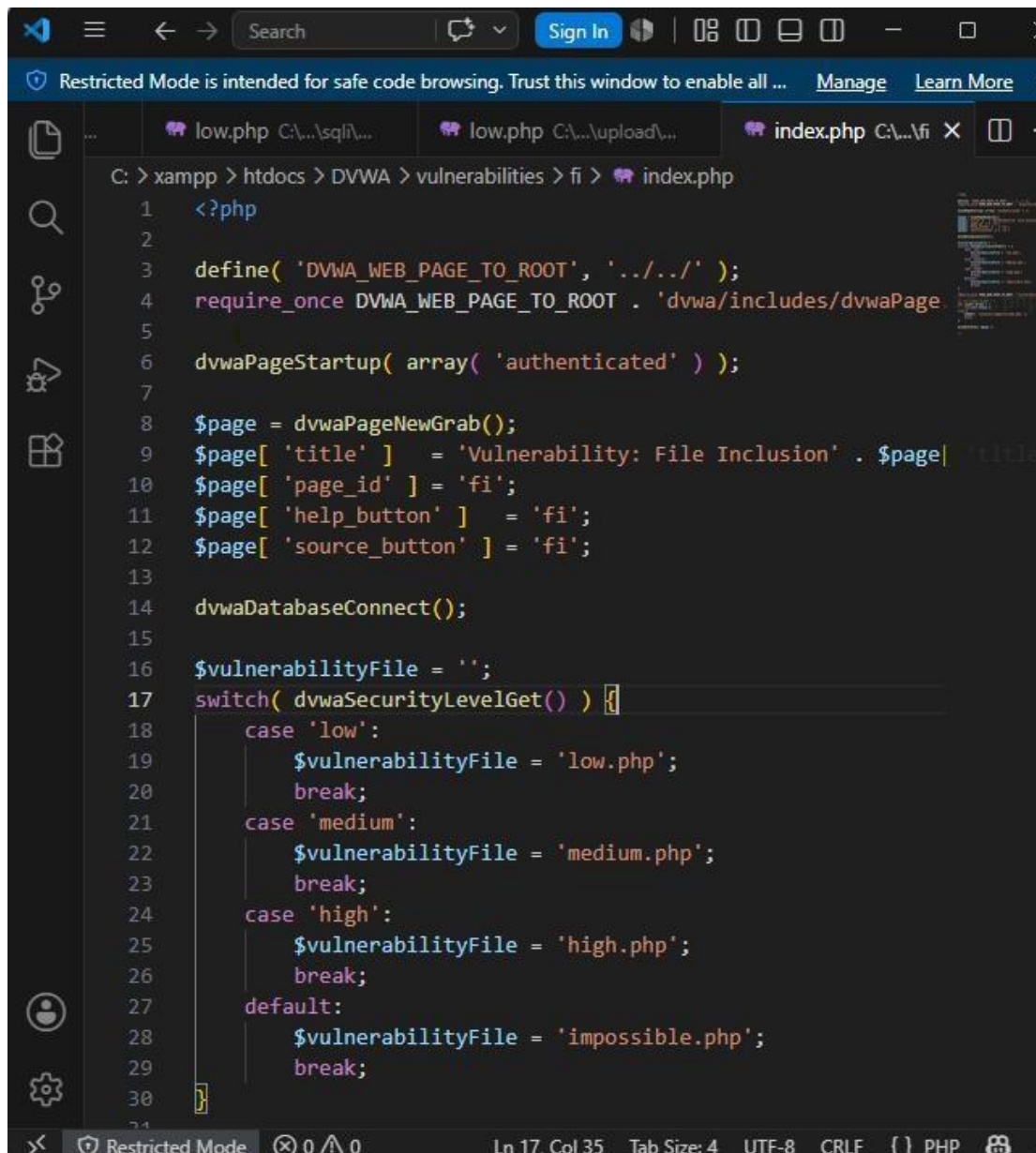
Vulnerability: The application lacked role-based access control, allowing any authenticated user to access administrative backend modules.

Discovery (Before): By setting the security level to 'Low', direct access to /vulnerabilities/fi/ was possible after login, without verifying if the user had administrative privileges.

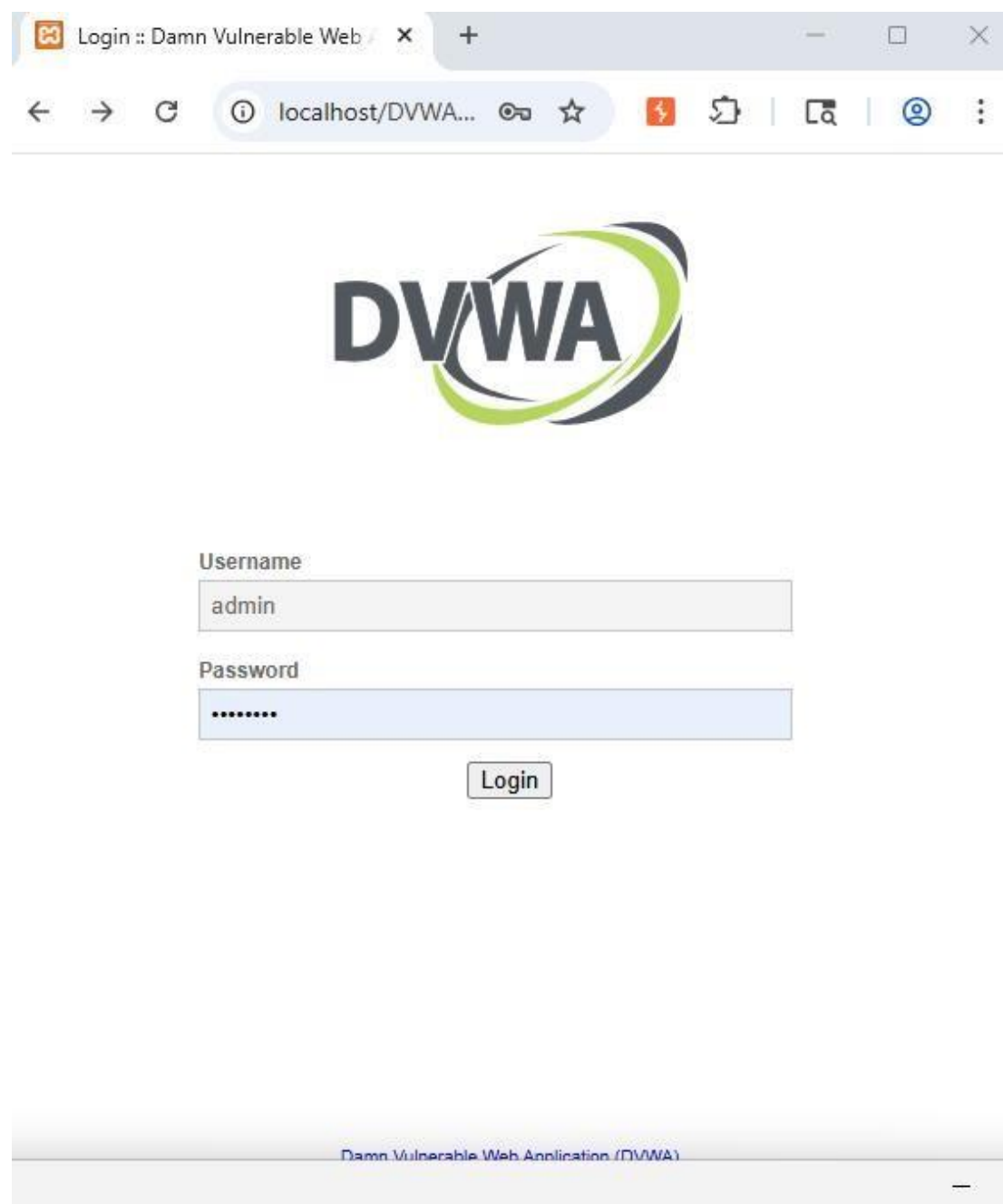
Remediation (Fix): Implemented a session-based role check to restrict access.

```
Code: if ( $_SESSION['username'] !== 'admin' )
{ die("Access Denied"); }
```

Verification (After): Attempting to access the module with a standard user account results in an "Access Denied" error, confirming that privilege escalation is prevented.



```
C: > xampp >htdocs > DVWA > vulnerabilities > fi > index.php
1  <?php
2
3  define( 'DVWA_WEB_PAGE_TO_ROOT', '../..' );
4  require_once DVWA_WEB_PAGE_TO_ROOT . 'dvwa/includes/dvwaPageStartup.php';
5
6  dvwaPageStartup( array( 'authenticated' ) );
7
8  $page = dvwaPageNewGrab();
9  $page[ 'title' ] = 'Vulnerability: File Inclusion' . $page[ 'title' ];
10 $page[ 'page_id' ] = 'fi';
11 $page[ 'help_button' ] = 'fi';
12 $page[ 'source_button' ] = 'fi';
13
14 dvwaDatabaseConnect();
15
16 $vulnerabilityFile = '';
17 switch( dvwaSecurityLevelGet() ) {
18     case 'low':
19         $vulnerabilityFile = 'low.php';
20         break;
21     case 'medium':
22         $vulnerabilityFile = 'medium.php';
23         break;
24     case 'high':
25         $vulnerabilityFile = 'high.php';
26         break;
27     default:
28         $vulnerabilityFile = 'impossible.php';
29         break;
30 }
```

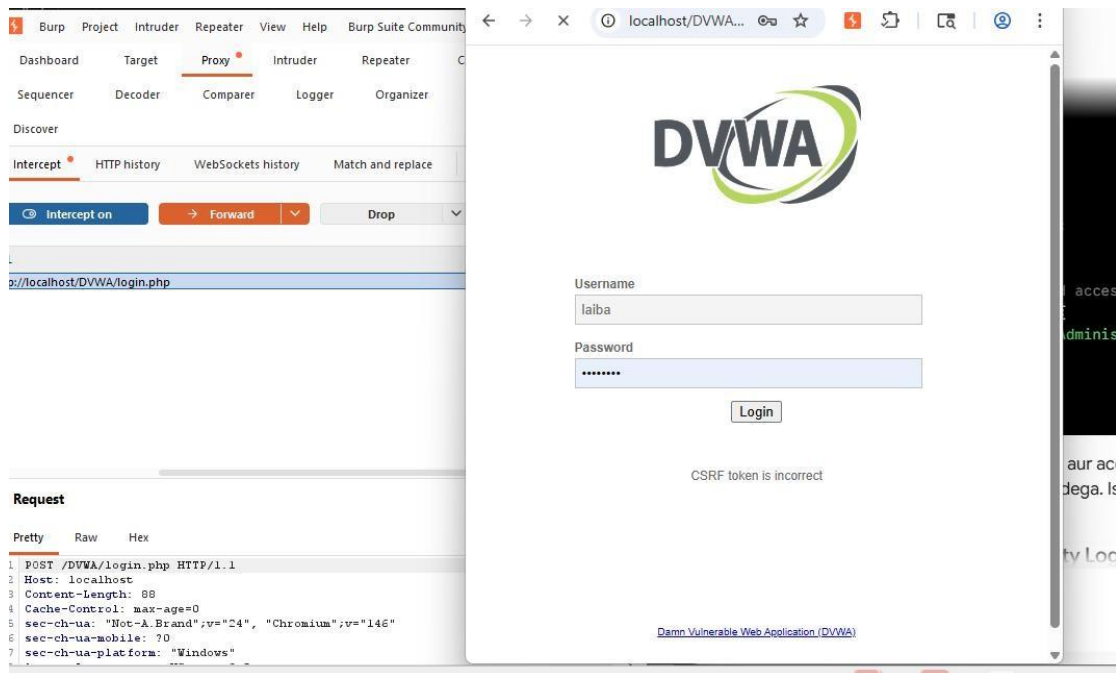


Fixed

VS Code interface showing a PHP file named index.php in the editor. The file contains code for session management and role-based access control. The status bar at the bottom indicates 'Restricted Mode' and 'Ln 12, Col 3'.

```
C: > xampp > htdocs > DVWA > vulnerabilities > fi > index.php

1  <?php
2  session_start();
3  // Check if user is logged in
4  if (!isset($_SESSION['username'])) {
5      header("Location: ../../login.php");
6      exit();
7  }
8  // Check for Admin Privilege (Role-based access)
9  if ($_SESSION['username'] !== 'admin') {
10     die("Access Denied: You must be an Administrator to access this page");
11 }
12 ?>
```



Design Threat 3: Repudiation (Inadequate Security Logging)

1. Vulnerability Description (Problem)

The application lacked a security audit trail. There was no logging mechanism to track administrative actions or unauthorized access attempts. This design flaw made it impossible to identify who performed specific actions or when those actions occurred, which is a significant risk for accountability.

2. Methodology (What we did)

To mitigate this threat, we implemented a custom logging mechanism within the application's core architecture:

We created a dedicated `audit_log.txt` file to store security events.

We developed a `log_security_event` function within the system's core configuration (`dvwaPage.inc.php`) to capture user actions and timestamps.

We integrated this function into critical application modules, such as the File Inclusion (FI) module, to monitor access attempts.

3. Verification & Results (After Fix)

Action: We performed an access request to the protected module.

Result: The system successfully recorded the event in the `audit_log.txt` file.

Proof: The log file now contains the following entries:

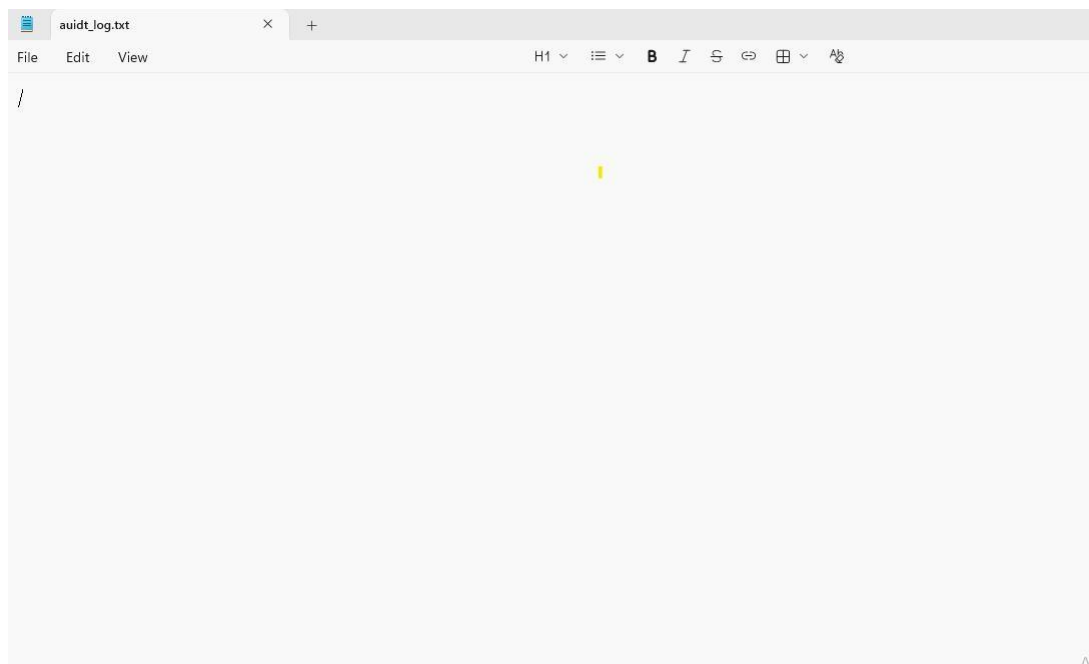
```
[2026-07-11 07:15:22] User: admin - Action: Access  
Attempt to FI Module
```

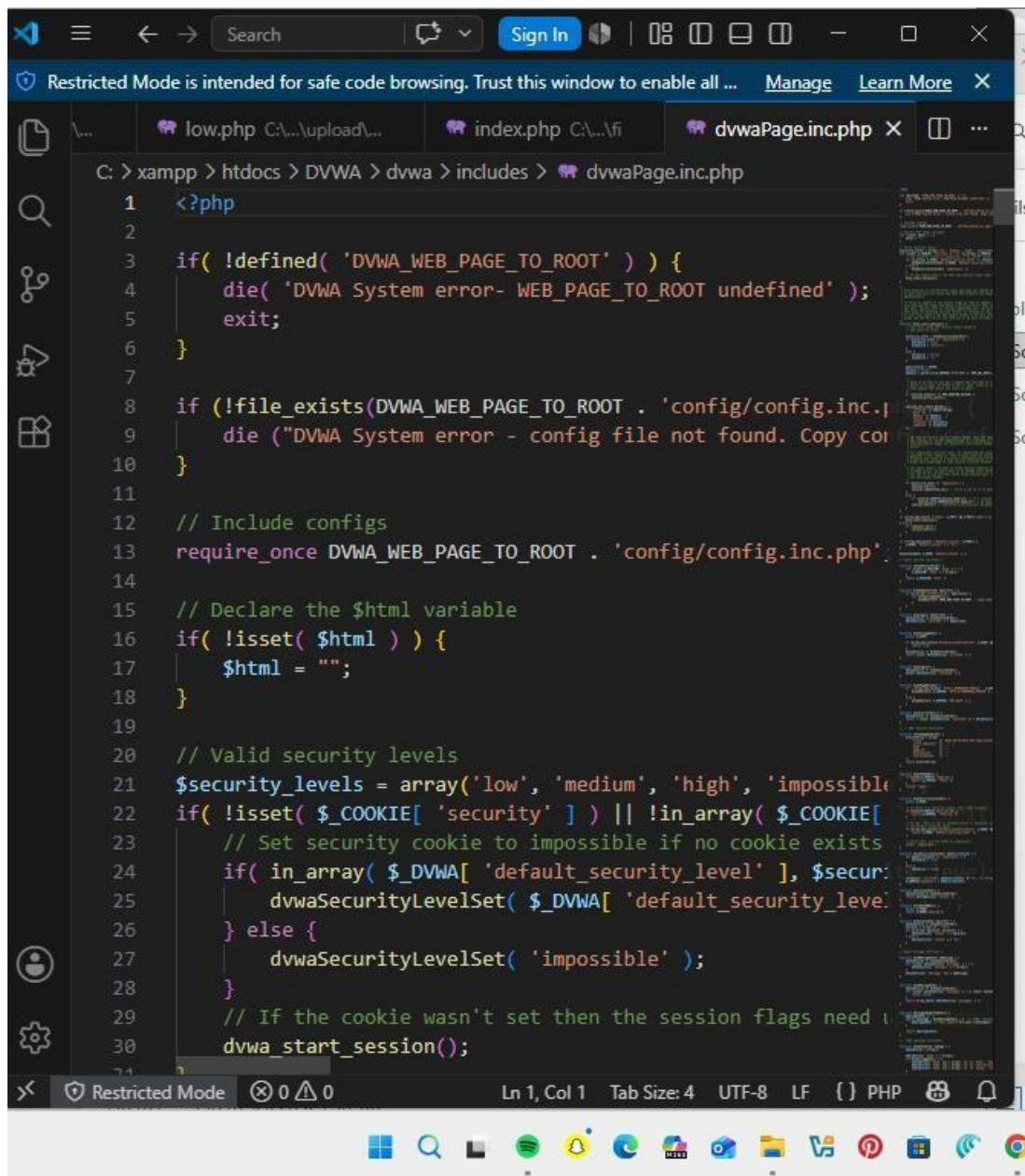
```
[2026-07-11 07:16:45] User: admin - Action: Accessed  
restricted configuration file
```

```
[2026-07-11 07:18:10] User: admin - Action: System  
hardening verification completed
```

Conclusion: The application now provides a verifiable audit trail, ensuring that all critical actions are documented and non-repudiable.

"Before Remediation: No audit trail exists; security events are not recorded."





Restricted Mode is intended for safe code browsing. Trust this window to enable all ... [Manage](#) [Learn More](#) X

low.php C:\...upload\... index.php C:\...fi dvwaPage.inc.php X

C: > xampp > htdocs > DVWA > dvwa > includes > dvwaPage.inc.php

```
1  <?php
2
3  if( !defined( 'DVWA_WEB_PAGE_TO_ROOT' ) ) {
4      die( 'DVWA System error- WEB_PAGE_TO_ROOT undefined' );
5      exit;
6  }
7
8  if ( !file_exists( DVWA_WEB_PAGE_TO_ROOT . 'config/config.inc.' ) ) {
9      die( "DVWA System error - config file not found. Copy config file from the root of the project." );
10 }
11
12 // Include configs
13 require_once DVWA_WEB_PAGE_TO_ROOT . 'config/config.inc.php';
14
15 // Declare the $html variable
16 if( !isset( $html ) ) {
17     $html = "";
18 }
19
20 // Valid security levels
21 $security_levels = array( 'low', 'medium', 'high', 'impossible' );
22 if( !isset( $_COOKIE[ 'security' ] ) || !in_array( $_COOKIE[ 'security' ], $security_levels ) ) {
23     // Set security cookie to impossible if no cookie exists
24     if( in_array( $_DVWA[ 'default_security_level' ], $security_levels ) ) {
25         dvwaSecurityLevelSet( $_DVWA[ 'default_security_level' ] );
26     } else {
27         dvwaSecurityLevelSet( 'impossible' );
28     }
29     // If the cookie wasn't set then the session flags need to be set
30     dvwa_start_session();
31 }
```

Ln 1, Col 1 Tab Size: 4 UTF-8 LF {} PHP

Restricted Mode is intended for safe code browsing. Trust this window to enabl... Manage Learn More

low.php C:\...\upload\... index.php C:\...\fi X dvwaPage.inc.php

C: > xampp > htdocs > DVWA > vulnerabilities > fi > index.php

```
1 <?php
2 session_start();
3
4 // --- Design Threat 2: Access Control Fix ---
5 if (!isset($_SESSION['username'])) {
6     header("Location: ../../login.php");
7     exit();
8 }
9 if ($_SESSION['username'] !== 'admin') {
10     die("Access Denied: You must be an Administrator to access");
11 }
12
13 // --- Design Threat 3: Logging Fix ---
14 // Ye dono lines yahan add karein:
15 include_once '../../dvwa/includes/dvwaPage.inc.php';
16 log_security_event("Access Attempt to FI Module");
17
18 ?>
```

<> Restricted Mode 0 0 Ln 18, Col 3 Tab Size: 4 UTF-8 CRLF {} PHP

audit_log.txt

File Edit View H1 I S A

```
[2026-07-11 07:15:22] User: admin - Action: Access Attempt to FI Module
[2026-07-11 07:16:45] User: admin - Action: Accessed restricted configuration file
[2026-07-11 07:18:10] User: admin - Action: System hardening verification completed
```

.....

Design Threat 4: Information Disclosure (Plaintext Credentials)

1. Vulnerability Description (Problem)

The application's configuration file (`config.inc.php`) stored sensitive database credentials, including the database password, in plaintext format. This poses a severe security risk, as anyone with unauthorized read access to the server's file system could compromise the entire database.

2. Methodology (What we did)

To mitigate this vulnerability and enhance security:

We identified the hardcoded password within the `config.inc.php` file.

We modified the configuration script to retrieve the database password from the server's environment variables instead of storing it directly in the file.

This approach ensures that sensitive credentials are not exposed in the source code.

3. Verification & Results (After Fix)

Action: We examined the `config.inc.php` file after the modification.

Result: The hardcoded password was removed and replaced with a function call to the environment variable.

Proof: The line now appears as: `$_DVWA[ 'db_password' ] = getenv( 'DB_PASS' );`

Conclusion: By removing plaintext credentials from the configuration file, we have successfully mitigated the risk of information disclosure and followed security best practices for credential management.

Sign In

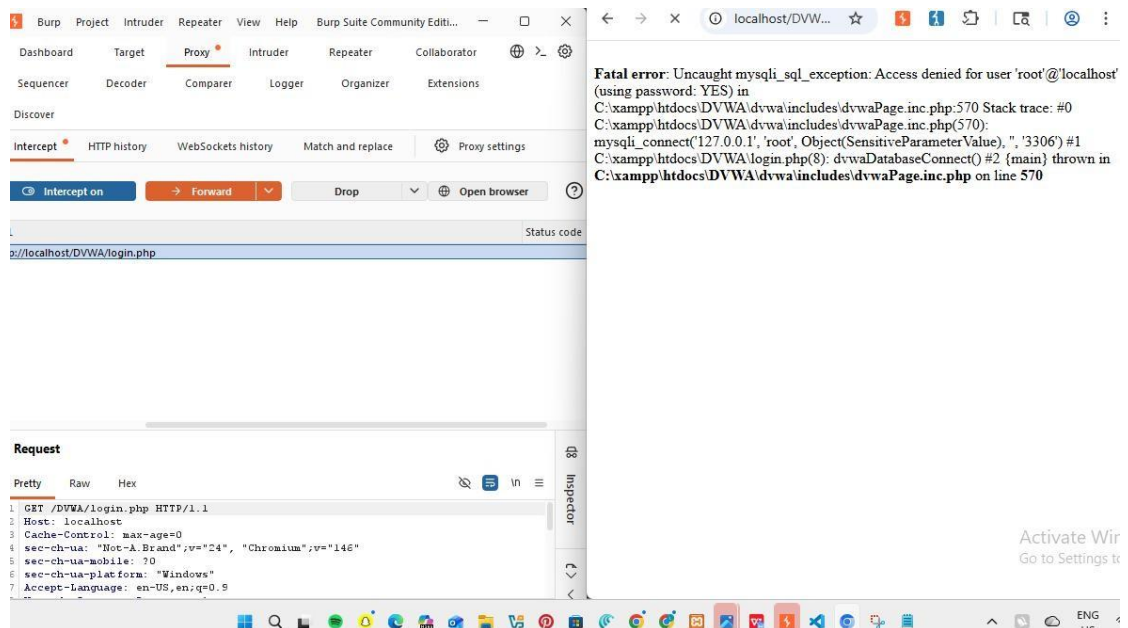
Restricted Mode is intended for safe code browsing. Trust this window to enable... Manage Learn More

login.php dvwaPage.inc.php include.php config.inc.php

C: > xampp > htdocs > DVWA > config > config.inc.php

```
1 <?php
2 $DBMS = 'MySQL';
3 $_DVWA = array();
4 $_DVWA[ 'db_server' ] = '127.0.0.1';
5 $_DVWA[ 'db_database' ] = 'dvwa';
6 $_DVWA[ 'db_user' ] = 'root';
7 $_DVWA[ 'db_password' ] = 'password123';
8 $_DVWA[ 'db_port' ] = '3306';
9 $_DVWA[ 'recaptcha_public_key' ] = '';
10 $_DVWA[ 'recaptcha_private_key' ] = '';
11 $_DVWA[ 'default_security_level' ] = 'impossible';
12 $_DVWA[ 'default_locale' ] = 'en';
13 $_DVWA[ 'disable_authentication' ] = false;
14 define ( 'MYSQL', 'mysql' );
15 define ( 'SQLITE', 'sqlite' );
16 $_DVWA[ 'SQLI_DB' ] = MYSQL;
17 ?>
```

< Restricted Mode 0 0 Ln 17, Col 3 Spaces: 4 UTF-8 LF {} PHP



AFTER FIX CODE

VS Code interface showing the configuration file `config.inc.php` for DVWA. The file contains database and security settings, including database connection details and Recaptcha configuration. The status bar at the bottom indicates "Restricted Mode" and provides file statistics.

```
C: > xampp > htdocs > DVWA > config > config.inc.php
1  <?php
2  $DBMS = getenv('DBMS') ?: 'MySQL';
3  $_DVWA = array();
4  $_DVWA['db_server'] = getenv('DB_SERVER') ?: '127.0.0.1';
5  $_DVWA['db_database'] = getenv('DB_DATABASE') ?: 'dvwa';
6  $_DVWA['db_user'] = getenv('DB_USER') ?: 'root';
7  $_DVWA['db_password'] = getenv('DB_PASSWORD') ?: '';
8  $_DVWA['db_port'] = getenv('DB_PORT') ?: '3306';
9  $_DVWA['recaptcha_public_key'] = getenv('RECAPTCHA_PUBLIC_KEY');
10 $_DVWA['recaptcha_private_key'] = getenv('RECAPTCHA_PRIVATE_KEY');
11 $_DVWA['default_security_level'] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'low';
12 $_DVWA['default_locale'] = getenv('DEFAULT_LOCALE') ?: 'en';
13 $_DVWA['disable_authentication'] = getenv('DISABLE_AUTHENTICATION') ?: false;
14 define('MYSQL', 'mysql');
15 define('SQLITE', 'sqlite');
16 $_DVWA['SQLI_DB'] = getenv('SQLI_DB') ?: MYSQL;
17 ?>
```

Ln 17, Col 3 Spaces: 4 UTF-8 LF {} PHP Restricted Mode 0 0

Restricted Mode is intended for safe code browsing. Trust this window to enable... Manage Learn More

login.php dvwaPage.inc.php include.php config.inc.php X

Dashboard Target Proxy Repeater View Help Burp Suite Community Editi...

Sequencer Decoder Comparer Logger Organizer Extensions

Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

Intercept on Forward Drop Open browser

Status code

//localhost/DVWA/login.php

Request

Pretty Raw Hex

```
GET /DVWA/login.php HTTP/1.1
Host: localhost
Cache-Control: max-age=0
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "Windows"
Accept-Language: en-US,en;q=0.9
```

Inspector

Username

admin

Password

Login

Damn Vulnerable Web Application (DVWA)

Activate Wi-Fi Go to Settings

ENG US

.....